

**TRƯỜNG KHOA HỌC TỰ NHIÊN, ĐHQG TP. HỒ CHÍ MINH**

**KHOA ĐIỆN TỬ - VIỄN THÔNG**

**THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT**

**ĐỒ ÁN**

**HỆ THỐNG BÃI GIỮ XE THÔNG MINH VÀ DẪN ĐƯỜNG**

**(SMART CAMPUS PARKING)**

GVHD: ThS Trần Tuấn Kiệt

Sinh viên: Lê Trần Khánh An, Nguyễn Huân, Nguyễn Tuấn Kiệt, Lê Minh Trọng, Lâm Trung Hiếu, Nguyễn Bá Thành, Mai Lý Khải Triều

MSSV: 22207001, 23207002, 23207008, 23207030, 23207054, 23207106, 23207136

# I. TÓM TẮT

## Mô tả chung:

- Hệ thống bãi giữ xe thông minh và dẫn đường (Smart Campus Parking) quy mô lớn trong môi trường trường học.
- Khắc phục rủi ro sai sót dữ liệu, ùn tắc cục bộ của phương pháp thủ công bằng cách tự động hóa điều phối giao thông nội khu, phân bổ không gian và xử lý nhiều dữ liệu.

## Dự án đáp ứng:

- Đồ thị (Graph) - Ma trận kề (305 đỉnh): Mô hình hóa sơ đồ bãi xe, lưu khoảng cách giữa các cổng và 300 vị trí đỗ (khu Giảng viên & Sinh viên).
- Thuật toán Dijkstra: Tính toán và tự động trích xuất lộ trình dẫn đường tối ưu, ngắn nhất từ cổng đến vị trí trống.
- Bảng băm (Hash Table) - Separate Chaining: Tối ưu hóa hoạt động kiểm soát xe ra/vào với thời gian truy xuất
- Hàng đợi (Queue - FIFO): Điều tiết luồng phương tiện tự động tại cổng khi bãi đỗ đạt sức chứa tối đa.
- Hàng đợi ưu tiên & Ngăn xếp (Stack): Xử lý linh hoạt bài toán chia sẻ không gian (xe sinh viên đỗ tạm khu giảng viên) và mô phỏng dời xe cản đường.
- Fuzzy Search - Levenshtein: Khắc phục sai số nhiều camera/mất thẻ bằng cách truy xuất 3 biến số có độ tương đồng cao nhất.

## Kết quả được hiển thị qua:

- C Console
- GUI

## II. CƠ SỞ LÝ THUYẾT

### 2.1. Danh sách liên kết đơn

Danh sách liên kết đơn là một cấu trúc dữ liệu tuyến tính động, được tạo thành từ tập hợp các phần tử nút. Các nút trong danh sách liên kết không nằm ở các vùng nhớ liên tiếp nhau mà phân tán ngẫu nhiên trong bộ nhớ Heap. Mỗi liên hệ thứ tự giữa các phần tử được duy trì thông qua các liên kết con trỏ một chiều.

Thuật toán Thêm\_Xe\_Vào\_Đầu\_Danh\_Sách(Tham chiếu List l, Xe x):

Đặt con trỏ p <- Tạo\_Nút\_Mới(x) {Gọi hàm GetNode cấp phát động}

Nếu p bằng NULL thì:

Trả về thất bại

Kết thúc nếu

Nếu l.first bằng NULL thì: {Trường hợp danh sách đang rỗng}

l.first <- p

l.last <- p

Ngược lại: {Danh sách đã có xe đỗ}

p.next <- l.first {Nối đuôi nút mới vào nút đầu hiện hành}

l.first <- p {Cập nhật nút đầu danh sách là nút mới}

Kết thúc nếu

Trả về thành công

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.2. Stack

Ngăn xếp là một cấu trúc vận hành theo cơ chế vào sau, ra trước. Dựa trên nguyên lý này, tiến trình bổ sung hoặc loại bỏ phần tử tuân thủ nghiêm ngặt quy tắc: phần tử được nạp vào cấu trúc muộn nhất sẽ là phần tử đầu tiên được phép giải phóng.

Thuật toán Push(Tham chiếu Stack S, Xe x):

```
{ Đầu vào: Cấu trúc quản lý Ngăn xếp S, thông tin xe x cần lưu trữ }
{ Đầu ra: Ngăn xếp S được cập nhật thêm nút mới chứa xe x ở vị trí đỉnh }
// Bước 1: Cấp phát động một ô nhớ mới trong bộ nhớ RAM
Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước StackNode
// Bước 2: Kiểm tra trạng thái bộ nhớ hệ thống
Nếu newNode bằng NULL thì:
    Thông báo lỗi "Lỗi hệ thống: Không đủ bộ nhớ RAM để cấp phát!"
    Trả về THẤT BẠI
Kết thúc nếu
// Bước 3: Nạp dữ liệu thực thể xe x vào nút mới
newNode.xe <- x
// Bước 4: Móc nối liên kết nút mới vào trên đỉnh Stack hiện tại
newNode.next <- S.top { Cho nút mới liên kết tới nút đang ở đỉnh hiện hành }
// Bước 5: Cập nhật con trỏ đỉnh quản lý nút mới
S.top <- newNode { Nút mới chính thức trở thành đỉnh Stack mới }
Trả về THÀNH CÔNG
Kết thúc thuật toán
```

## II. CƠ SỞ LÝ THUYẾT

### 2.3. Hàng đợi (Queue)

Hàng đợi là một cấu trúc vận hành theo cơ chế vào trước, ra trước. Dựa trên nguyên lý này, tiến trình bổ sung hoặc loại bỏ phần tử tuân thủ quy tắc: phần tử được nạp vào cấu trúc sớm nhất sẽ là phần tử đầu tiên được phép giải phóng.

Thuật toán EnQueue(Tham chiếu Queue Q, Xe x):

{ Đầu vào: Cấu trúc quản lý Hàng đợi Q, thực thể dữ liệu xe x cần thêm }

{ Đầu ra: Hàng đợi Q được cập nhật thêm nút mới chứa xe x ở cuối xâu }

// Bước 1: Cấp phát động một ô nhớ mới trong bộ nhớ RAM

Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước QueueNode

// Bước 2: Kiểm tra trạng thái cấp phát của bộ nhớ hệ thống

Nếu newNode bằng NULL thì:

    Thông báo lỗi "Lỗi hệ thống: Không đủ bộ nhớ RAM để cấp phát!"

    Dừng thuật toán

Kết thúc nếu

// Bước 3: Đóng gói thông tin xe x và khóa mới liên kết kế tiếp

newNode.xe <- x

newNode.next <- NULL

// Bước 4: Đưa nút mới vào cuối hàng đợi dựa trên trạng thái rỗng

Nếu Q.rear bằng NULL (Hàng đợi đang rỗng) thì:

    Q.front <- newNode     { Nút mới đóng vai trò vừa là đầu vừa là cuối }

    Q.rear <- newNode

Ngược lại (Hàng đợi đã có sẵn các xe xếp hàng):

    Q.rear.next <- newNode { Móc nối nút cuối hiện hành tới nút mới }

    Q.rear <- newNode { Dịch chuyển con trỏ rear quản lý nút mới }

Kết thúc nếu

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.4. Bảng băm (Hash Table)

- Bảng băm (Hash Table) là một cấu trúc được thiết kế nhằm tối ưu hóa tốc độ thực thi của các thao tác chèn, tra cứu và loại bỏ các cặp khóa - giá trị.
- Cấu trúc này vận hành dựa trên nguyên lý băm, sử dụng một hàm băm để chuyển đổi khóa thành một chỉ số tương ứng trong mảng.
- Chỉ số này định vị chính xác vị trí lưu trữ trong bộ nhớ (bucket) để liên kết trực tiếp với giá trị cần quản lý.
- Cơ chế này cho phép truy xuất trực tiếp đến phân vùng dữ liệu đích mà không cần thực hiện tiến trình duyệt tuần tự, từ đó tối ưu hóa độ phức tạp thời gian của các thao tác.

Thuật toán hashFunction(Chuỗi key):

```
{ Đầu vào: Chuỗi ký tự key đại diện cho Mã thẻ xe }
{ Đầu ra: Số nguyên index đại diện cho chỉ mục ô chứa (bucket) }
Đặt biến hash <- 0
Đặt biến i <- 0
// Duyệt qua từng ký tự của chuỗi cho đến khi gặp ký tự kết thúc chuỗi
Trong khi key[i] khác ký tự rỗng ('\0') thực hiện:
    hash <- (hash * 31) + mã_ASCII_của(key[i]) { Thuật toán băm đa thức nhân cơ số 31
}
    i <- i + 1
Kết thúc vòng lặp
Trả về (hash chia lấy dư cho HASH_SIZE)      { Phép toán Modulo giới hạn chỉ mục }
Kết thúc thuật toán
```

## II. CƠ SỞ LÝ THUYẾT

### 2.5. Separate Chaining

- Separate Chaining là kỹ thuật giải quyết xung đột trong cấu trúc dữ liệu Bảng băm.
- Hiện tượng xung đột phát sinh khi hàm băm ánh xạ hai hoặc nhiều khóa phân biệt về cùng một chỉ số của mảng lưu trữ nền tảng.
- Thay vì tìm kiếm một vị trí trống khác trên mảng, phương pháp này xử lý xung đột bằng cách tổ chức mỗi phần tử của mảng (ô chứa) thành một con trỏ quản lý một danh sách liên kết độc lập.
- Khi xảy ra xung đột, phần tử mới sẽ được cấp phát bộ nhớ động và nối tiếp vào danh sách liên kết tại chỉ số mảng tương ứng, giúp bảo toàn các cặp khóa - giá trị mà không làm thay đổi hay chiếm dụng không gian lưu trữ của các vị trí khác trong bảng băm.
-

## II. CƠ SỞ LÝ THUYẾT

### 2.5. Separate Chaining

Thuật toán insertHash(Tham chiếu HashTable HT, Xe xe\_moi):

// Bước 1: Tính chỉ mục ô chứa bằng cách băm Mã thẻ xe

Đặt số nguyên idx <- hashFunction(xe\_moi.maThe)

// Bước 2: Cấp phát động một nút băm mới trong RAM

Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước HashNode

Nếu newNode bằng NULL thì:

    Thông báo lỗi "Lỗi: Không đủ bộ nhớ RAM!"

    Dừng thuật toán

Kết thúc nếu

// Bước 3: Đóng gói dữ liệu xe mới vào nút băm

newNode.xe <- xe\_moi

// Bước 4: Áp dụng kỹ thuật Separate Chaining (Chèn vào đầu danh sách liên kết)

newNode.next <- HT.buckets[idx]    { Nút mới trỏ tới nút đầu hiện tại của thùng idx }

HT.buckets[idx] <- newNode{ô idx cập nhật đầu danh sách mới là newNode }

Kết thúc thuật toán



## II. CƠ SỞ LÝ THUYẾT

### 2.6. Ma trận kề

- Ma trận kề biểu diễn cấu trúc dữ liệu đồ thị trong bộ nhớ dưới dạng một mảng hai chiều (ma trận vuông) có kích thước  $V \times V$ , với  $V$  là tổng số lượng đỉnh của đồ thị.
- Trong ma trận kề, cả hàng và cột đều đại diện cho danh sách các đỉnh của đồ thị. Giá trị nằm tại ô giao giữa hàng  $i$  và cột  $j$  (ký hiệu là  $matrix[i][j]$ ) sẽ biểu diễn thông tin liên quan đến cạnh nối từ đỉnh  $i$  tới đỉnh  $j$ .

## II. CƠ SỞ LÝ THUYẾT

### 2.6. Ma trận kề

Thuật toán loadMapFromFile(Tham chiếu Graph G, Chuỗi ten\_file):

{ Đầu vào: Cấu trúc đồ thị G, tên file cấu hình bản đồ (ví dụ: "map.txt") }

{ Đầu ra: Đồ thị G được điền đầy đủ dữ liệu phân khu và ma trận kề khoảng cách }

// Bước 1: Mở tệp tin văn bản ở chế độ chỉ đọc ("rt")

Đặt con trỏ file f <- Mở file ten\_file ở chế độ "rt"

// Bước 2: Kiểm tra sự tồn tại của tệp tin bản đồ

Nếu f bằng NULL thì:

Thông báo lỗi "Lỗi hệ thống: Không thể mở tệp tin bản đồ!"

Dừng thuật toán

Kết thúc nếu

// Bước 3: Đọc dòng đầu tiên để xác định số lượng đỉnh (G.n) của sa bàn

Đọc từ file f ghi vào G.n

// Bước 4: Đọc thông tin phân loại khu vực cho từng đỉnh

Cho i chạy từ 0 đến G.n - 1 thực hiện:

Đọc từ file f ghi vào G.zoneType[i] {-1: Cổng, 1: Khu GV, 0: Khu SV}

G.isOccupied[i] <- 0 { Khởi tạo mặc định: tất cả ô đồ đều TRỐNG }

Kết thúc vòng lặp

G.isOccupied <- 1 { Đỉnh 0 là Cổng vào, mặc định luôn khóa rào chắn }

// Bước 5: KHỞI TẠO MA TRẬN KÊ HAI CHIỀU (Đọc trọng số khoảng cách giữa các đỉnh)

Cho i chạy từ 0 đến G.n - 1 thực hiện:

Cho j chạy từ 0 đến G.n - 1 thực hiện:

Đọc từ file f ghi vào G.matrix[i][j] { Gán khoảng cách từ đỉnh i tới đỉnh j }

Kết thúc vòng lặp j

Kết thúc vòng lặp i

// Bước 6: Đóng tệp tin sau khi hoàn tất nạp dữ liệu vào RAM

Đóng file f

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.7. Hàng đợi ưu tiên (Priority Queue)

- Hàng đợi ưu tiên (Priority Queue) là cấu trúc trong đó mỗi phần tử lưu trữ đều được gắn liền với một giá trị ưu tiên. Khác với hàng đợi thông thường hoạt động nghiêm ngặt theo cơ chế FIFO (Vào trước ra trước), các phần tử trong hàng đợi ưu tiên được truy xuất và phục vụ hoàn toàn dựa trên mức độ ưu tiên của chúng:
  - Các phần tử có mức độ ưu tiên cao hơn sẽ luôn được truy xuất hoặc loại bỏ ra khỏi hàng đợi trước các phần tử có mức độ ưu tiên thấp hơn, bất kể trình tự chúng được đưa vào hệ thống
  - Khi một phần tử mới được thêm vào, cấu trúc sẽ tự động sắp xếp và chèn phần tử đó vào vị trí tương ứng với mức độ ưu tiên vốn có của nó

## II. CƠ SỞ LÝ THUYẾT

### 2.7. Hàng đợi ưu tiên (Priority Queue)

Thuật toán pushPQ(Tham chiếu PriorityQueue PQ, Xe xe\_moi):

{ Đầu vào: Cấu trúc hàng đợi ưu tiên PQ, thực thể xe\_moi cần chèn }

{ Đầu ra: Hàng đợi PQ được cập nhật, xe\_moi được đưa về đúng vị trí vun đống }

// Bước 1: Kiểm tra dung lượng bộ nhớ mảng Heap

Nếu PQ.size bằng PQ.capacity thì:

PQ.capacity <- PQ.capacity \* 2 { Nhân đôi sức chứa tối đa }

Cấp phát lại bộ nhớ (Reallocate) cho PQ.heap với sức chứa mới

Nếu PQ.heap bằng NULL thì:

Thông báo lỗi "Lỗi hệ thống: Không thể tái cấp phát bộ nhớ Heap!"

Dừng thuật toán

Kết thúc nếu

Kết thúc nếu

// Bước 2: Chèn tạm thời xe mới vào vị trí cuối cùng của mảng Heap

PQ.heap[PQ.size] <- xe\_moi

Đặt số nguyên i <- PQ.size { Lưu vết chỉ số hiện tại của xe mới }

PQ.size <- PQ.size + 1 { Tăng số lượng xe hiện hành }

// Bước 3: Thuật toán Vun đống ngược lên (Up-Heapify / Shift-Up)

// Trong khi i chưa phải là gốc (i > 0) VÀ xe hiện tại vào bãi SỚM HƠN xe cha của nó

Trong khi (i > 0) VÀ (PQ.heap[i].thoiGianVao < PQ.heap[(i - 1) / 2].thoiGianVao) thực hiện:

Hoán\_Vị(PQ.heap[i], PQ.heap[(i - 1) / 2]) { Đổi chỗ xe con và xe cha }

i <- (i - 1) / 2 { Dịch chuyển chỉ số i lên vị trí nút cha }

Kết thúc vòng lặp

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.8. Heap (Min Heap)

Min-Heap là một cấu trúc dữ liệu phi tuyến tính dựa trên mô hình cây nhị phân hoàn chỉnh. Trong cấu trúc Min-Heap, phần tử tại mỗi nút cha luôn luôn có giá trị nhỏ hơn hoặc bằng giá trị lưu trữ tại các nút con trực tiếp của nó

Thuật toán Min\_Heapify(Mảng arr, Số nguyên r, Số nguyên i):

{ Đầu vào: Mảng arr, giới hạn phạm vi hiệu chỉnh r, chỉ số nút cần xét i }

{ Đầu ra: Nhánh cây có gốc tại i được hiệu chỉnh thỏa mãn tính chất Min-Heap }

Đặt số nguyên smallest <- i { Giả định nút cha hiện tại là nhỏ nhất }

Đặt số nguyên index <- 2 \* i + 1 { Xác định chỉ số nút con bên trái }

// Bước 1: So sánh hai nút con để tìm nút con có giá trị nhỏ nhất

Nếu (index + 1 < r) VÀ (arr[index] > arr[index + 1]) thì:

index <- index + 1 { Chọn nút con bên phải vì nó nhỏ hơn con trái }

Kết thúc nếu

// Bước 2: So sánh nút con nhỏ nhất vừa chọn với nút cha (smallest)

Nếu (index < r) VÀ (arr[index] < arr[smallest]) thì:

smallest <- index { Cập nhật chỉ số của phần tử nhỏ nhất }

Kết thúc nếu

// Bước 3: Nếu nút cha lớn hơn nút con (vi phạm tính chất Min-Heap)

Nếu smallest khác i thì:

Hoán\_Vị(arr[i], arr[smallest]) { Đổi chỗ nút cha và nút con nhỏ nhất }

Gọi đệ quy Min\_Heapify(arr, r, smallest) { Lan truyền hiệu chỉnh xuống nhánh con }

Kết thúc nếu

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.9. Quy hoạch động (Dynamic Programming)

- Quy hoạch động là một phương pháp thiết kế giải thuật vận hành bằng cách chia bài toán chính thành các bài toán con, tìm lời giải cho các bài toán con đó rồi tổng hợp lại để hình thành lời giải hoàn chỉnh.
- Để có thể áp dụng phương pháp này, bài toán cần giải quyết bắt buộc phải hội tụ đủ hai thuộc tính cấu trúc:
  - Các bài toán con trùng lặp (lời giải của bài toán con này là một phần cấu thành nên lời giải của bài toán con khác)
  - Cấu trúc con tối ưu (lời giải tối ưu của bài toán chính được xây dựng từ lời giải tối ưu của các bài toán con nhỏ hơn).

## II. CƠ SỞ LÝ THUYẾT

### 2.10. Khoảng cách Levenshtein

- Khoảng cách Levenshtein là một thuật toán đo lường sai khác giữa hai chuỗi ký tự bằng cách xác định số lượng thao tác tối thiểu để biến đổi chuỗi ban đầu thành chuỗi mục tiêu.
- Các thao tác hợp lệ được định nghĩa ở cấp độ đơn ký tự, bao gồm ba phép biến đổi nguyên tử: chèn (bổ sung một ký tự), xóa (loại bỏ một ký tự), và thay thế (đổi một ký tự này thành một ký tự khác).

## II. CƠ SỞ LÝ THUYẾT

### 2.10. Khoảng cách Levenshtein

Thuật toán getLevenshteinDistance(Chuỗi s1, Chuỗi s2) bằng Quy hoạch động:

{ Đầu vào: Hai chuỗi ký tự s1 (độ dài M) và s2 (độ dài N) đại diện cho hai biến số }

{ Đầu ra: Số nguyên là khoảng cách chỉnh sửa tối thiểu giữa hai chuỗi biến số }

Đặt M <- Độ\_Dài(s1)

Đặt N <- Độ\_Dài(s2)

// Khởi tạo bảng phương án Quy hoạch động kích thước (M + 1) x (N + 1) trong bộ nhớ RAM

Đặt mảng hai chiều dp kích thước (M + 1) x (N + 1) số nguyên

// Bước 1: Khởi tạo các bài toán con cơ sở (Base Cases)

Cho i chạy từ 0 đến M thực hiện:

dp[i] <- i { Biến đổi tiền tố s1[0..i-1] thành chuỗi rỗng bằng phép xóa i ký tự }

Kết thúc vòng lặp

Cho j chạy từ 0 đến N thực hiện:

dp[j] <- j { Biến đổi chuỗi rỗng thành tiền tố s2[0..j-1] bằng phép chèn j ký tự }

Kết thúc vòng lặp

// Bước 2: Điền bảng phương án theo cơ chế từ dưới lên (Bottom-up Tabulation)  
Cho i chạy từ 1 đến M thực hiện:

Cho j chạy từ 1 đến N thực hiện:

// Xác định chi phí của phép thay thế ký tự (cost)

Nếu s1[i - 1] bằng s2[j - 1] thì:

Đặt cost <- 0 { Hai ký tự trùng khớp nhau, không tốn chi phí thay đổi }

Ngược lại:

Đặt cost <- 1 { Ký tự khác nhau, tốn 1 phép thay thế }

Kết thúc nếu

// Áp dụng công thức chuyển trạng thái Quy hoạch động để tìm giá trị tối ưu

Đặt xoa <- dp[i - 1][j] + 1 { Phép xóa 1 ký tự }

Đặt chen <- dp[i][j - 1] + 1 { Phép chèn 1 ký tự }

Đặt thay\_the <- dp[i - 1][j - 1] + cost { Phép thay thế 1 ký tự }

// Lưu kết quả tối ưu của bài toán con vào ô nhớ tương ứng

dp[i][j] <- Giá\_Trị\_Nhỏ\_Nhất\_Trong\_Ba(xoa, chen, thay\_the)

Kết thúc vòng lặp j

Kết thúc vòng lặp i

Trả về dp[M][N] { Kết quả tối ưu của bài toán gốc cần tìm }

Kết thúc thuật toán



## II. CƠ SỞ LÝ THUYẾT

### 2.11. Bubble Sort

- Bubble Sort là thuật toán sắp xếp bằng so sánh và đổi chỗ các cặp phần tử kế cận nhau nếu chúng đứng sai thứ tự (nghịch thế).
- Thao tác này giúp đưa phần tử nhỏ (hoặc lớn) hơn trong cặp đang xét về vị trí đứng đầu (hoặc cuối) dãy hiện hành.
- Thuật toán lặp lại xử lý trên cho đến khi không còn cặp phần tử nào vi phạm.
- Sau mỗi lượt duyệt, các phần tử tối ưu tiếp theo sẽ lần lượt được nổi lên về đúng vị trí của chúng, đồng thời phạm vi dãy cần xét ở lượt kế tiếp sẽ được co ngắn lại.

## II. CƠ SỞ LÝ THUYẾT

### 2.11. Bubble Sort

Thuật toán BubbleSortFuzzy(Mảng arr, Số nguyên n):

{ Đầu vào: Bảng arr gồm n phần tử kiểu FuzzyResult cần sắp xếp }

{ Đầu ra: Bảng arr được sắp xếp theo thứ tự score tăng dần (độ lệch nhỏ nhất lên đầu) }

Đặt số nguyên i <- 0

Đặt số nguyên j <- 0

// Vòng lặp ngoài co ngắn dần phân vùng cần sắp xếp từ đầu mảng [2]

Cho i chạy từ 0 đến n - 2 thực hiện:

// Vòng lặp trong duyệt ngược từ cuối dãy về vị trí i hiện hành [2]

Cho j chạy từ n - 1 giảm dần về i + 1 thực hiện:

// Nếu phát hiện nghịch thế: xe đứng sau có độ lệch ký tự (score) nhỏ hơn xe đứng trước

Nếu  $arr[j].score < arr[j - 1].score$  thì:

Hoán\_Vị( $arr[j]$ ,  $arr[j - 1]$ ) { Đổi chỗ hai thực thể kết quả }

Kết thúc nếu

Kết thúc vòng lặp j

Kết thúc vòng lặp i

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.12. Fuzzy Search

- Fuzzy Search hay so khớp chuỗi xấp xỉ dùng để nhận diện các phần tử văn bản, chuỗi ký tự hoặc mục nhập có mức độ tương đồng cao nhưng không nhất thiết phải trùng khớp hoàn toàn .
- Thuật toán này được áp dụng phổ biến khi hệ thống không thể tìm thấy kết quả khớp chính xác tuyệt đối cho một từ khóa hay truy vấn trong cơ sở dữ liệu. Lúc này, thuật toán sẽ đối chiếu và trả về các kết quả có tỷ lệ tương đồng lớn hơn một ngưỡng được xác định trước.

## II. CƠ SỞ LÝ THUYẾT

### 2.12. Fuzzy Search

Thuật toán fuzzySearchLicensePlate(HashTable HT, Chuỗi queryPlate):

{ Đầu vào: Bảng băm HT chứa các xe đỗ trong bãi, chuỗi truy vấn queryPlate }

{ Đầu ra: Mảng chứa tối đa 3 chiếc xe có độ tương đồng biến số cao nhất }

// Bước 1: Chuẩn hóa chuỗi truy vấn đầu vào (viết hoa, loại bỏ ký tự đặc biệt)

Đặt chuỗi normQuery

Gọi normalizeString(queryPlate, normQuery)

// Bước 2: Khởi tạo danh sách ứng viên tạm để chứa kết quả khớp

Đặt mảng candidateList gồm 1000 phần tử kiểu FuzzyResult

Đặt số nguyên count <- 0

// Bước 3: Duyệt toàn bộ bảng băm (Separate Chaining) để tìm xe

Cho i chạy từ 0 đến HASH\_SIZE - 1 thực hiện:

Đặt con trỏ chạy curr <- HT.buckets[i]

Trong khi curr khác NULL thực hiện:

// Chuẩn hóa biến số của chiếc xe đang xét trong bãi

Đặt chuỗi normPlate

Gọi normalizeString(curr.xe.bienSo, normPlate)

// Tính điểm số sai lệch bằng thuật toán khoảng cách Levenshtein

Đặt số nguyên dist <- getLevenshteinDistance(normQuery, normPlate)

// Ghi nhận ứng viên vào danh sách tạm

candidateList[count].xe <- curr.xe

candidateList[count].score <- dist

count <- count + 1

curr <- curr.next

Kết thúc vòng lặp

Kết thúc vòng lặp

// Bước 4: Sắp xếp danh sách ứng viên theo score tăng dần (lệch ít nhất lên đầu)

Gọi BubbleSortFuzzy(candidateList, count)

// Bước 5: Trích xuất ra tối đa 3 chiếc xe có độ tương đồng cao nhất

Đặt mảng topResults gồm 3 phần tử kiểu Xe

Đặt số nguyên limit <- Giá\_Trị\_Nhỏ\_Nhất(3, count)

Cho i chạy từ 0 đến limit - 1 thực hiện:

topResults[i] <- candidateList[i].xe

Kết thúc vòng lặp

Trả về topResults

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.13. Shortest Path Dijkstra

- Trong lý thuyết đồ thị, bài toán tìm đường đi ngắn nhất là bài toán tìm kiếm một lộ trình di chuyển giữa hai đỉnh trên một đồ thị có trọng số sao cho tổng trọng số của các cạnh cấu thành nên lộ trình đó là nhỏ nhất. Tổng trọng số tích lũy dọc theo các cạnh của lộ trình di chuyển được gọi là chi phí đường đi hoặc trọng số đường đi. Đồ thị biểu diễn bài toán này có thể mô phỏng rất nhiều mô hình thực tế, trong đó các đỉnh đại diện cho các địa điểm giao lộ, ô đỗ xe, hoặc bộ định tuyến, còn các cạnh đại diện cho các con đường kết nối vật lý

## II. CƠ SỞ LÝ THUYẾT

### 2.13. Shortest Path Dijkstra

Thuật toán findNearestSlot(Đồ thị G, Số nguyên startNode, Số nguyên isGiangVien, Tham chiếu Số nguyên pathLength):

```
{ Đầu vào: Đồ thị G (chứa ma trận kề matrix, thông tin khu vực zoneType, trạng thái isOccupied) }  
{ startNode (Cổng vào = 0), isGiangVien (Phân khu yêu cầu: 1: Khu GV, 0: Khu SV) }  
{ Đầu ra: Mảng chứa lộ trình di chuyển tối ưu từ cổng đến ô đỗ gần nhất }  
// Bước 1: Khởi tạo các mảng hỗ trợ lưu trữ trạng thái duyệt  
Đặt mảng dist gồm G.n phần tử kiểu Số nguyên { Lưu khoảng cách ngắn nhất tạm thời từ nguồn }  
Đặt mảng visited gồm G.n phần tử kiểu Logic { Đánh dấu các đỉnh đã được duyệt xong }  
Đặt mảng trace gồm G.n phần tử kiểu Số nguyên { Lưu vết đỉnh cha đứng ngay trước (tiền bối) }
```

Cho i chạy từ 0 đến G.n - 1 thực hiện:

```
dist[i] <- INF { Mặc định khoảng cách ban đầu tới tất cả các đỉnh là vô cùng }  
visited[i] <- False { Tất cả các đỉnh đều được đánh dấu chưa duyệt }  
trace[i] <- -1 { Chưa ghi nhận vết di chuyển }
```

Kết thúc vòng lặp

```
dist[startNode] <- 0 { Thiết lập khoảng cách từ cổng tới chính nó bằng 0 }
```

```
Đặt số nguyên targetNode <- -1 { Biến tạm chứa ID ô đỗ tối ưu được chọn }
```

## II. CƠ SỞ LÝ THUYẾT

### 2.13. Shortest Path Dijkstra

// Bước 2: Vòng lặp chính của thuật toán Dijkstra (Duyệt tối đa G.n lần)

Lặp G.n lần thực hiện:

// 2.1. Tìm đỉnh u chưa duyệt có khoảng cách ngắn nhất đến nguồn

Đặt số nguyên min\_dist <- INF

Đặt số nguyên u <- -1

Cho i chạy từ 0 đến G.n - 1 thực hiện:

Nếu (visited[i] bằng False) VÀ (dist[i] < min\_dist) thì:

min\_dist <- dist[i]

u <- i

Kết thúc nếu

Kết thúc vòng lặp

// Nếu không tìm thấy đỉnh u thích hợp hoặc các đỉnh còn lại đều không thể chạm tới

Nếu (u bằng -1) HOẶC (dist[u] bằng INF) thì:

Thoát vòng lặp chính

Kết thúc nếu

// Đánh dấu đỉnh u đã hoàn tất xử lý (visited)

visited[u] <- True

// CƠ CHẾ DỪNG SỚM CHỈ CÓ TRONG ĐỒ ÁN

// Nếu đỉnh u được chọn là một ô trống TRỐNG và thuộc đúng phân khu quy định

Nếu (u khác startNode) VÀ (G.isOccupied[u] bằng 0) VÀ (G.zoneType[u] bằng isGiangVien) thì:

targetNode <- u { Ghi nhận u chính là ô trống gần cổng nhất }

Thoát vòng lặp chính { Dừng thuật toán sớm để tiết kiệm chi phí tính toán }

Kết thúc nếu

## II. CƠ SỞ LÝ THUYẾT

### 2.13. Shortest Path Dijkstra

// 2.2. Tiến hành nới lỏng (Relaxation) khoảng cách tới các đỉnh v kề với u

Cho v chạy từ 0 đến  $G.n - 1$  thực hiện:

// Nếu tồn tại đường đi trực tiếp giữa u và v và v chưa duyệt xong

Nếu ( $G.matrix[u][v]$  khác INF) VÀ ( $visited[v]$  bằng False) thì:

Đặt số nguyên  $alt \leftarrow dist[u] + G.matrix[u][v]$

// Nếu tìm được lộ trình mới qua u ngắn hơn khoảng cách cũ đã lưu của v

Nếu  $alt < dist[v]$  thì:

$dist[v] \leftarrow alt$

$trace[v] \leftarrow u$  { Lưu vết: Đỉnh đứng trước v trong lộ trình ngắn nhất là u }

Kết thúc nếu

Kết thúc nếu

Kết thúc vòng lặp

Kết thúc vòng lặp



## II. CƠ SỞ LÝ THUYẾT

### 2.13. Shortest Path Dijkstra

// Bước 3: Rebuild lộ trình di chuyển (Path Tracing) bằng cách đi lùi mảng tiền bối

Nếu targetNode khác -1 thì:

Đặt mảng path gồm G.n phần tử kiểu Số nguyên

Đặt số nguyên temp <- targetNode

pathLength <- 0

Trong khi temp khác -1 thực hiện:

path[pathLength] <- temp

pathLength <- pathLength + 1

temp <- trace[temp] { Đi ngược về đỉnh cha đứng trước }

Kết thúc vòng lặp

Trả về path { Trả về mảng chứa lộ trình ngược (Đích -> Cổng) }

Ngược lại:

Trả về NULL { Báo lỗi: Không tìm thấy ô đổ trống phù hợp }

Kết thúc nếu

Kết thúc thuật toán

## II. CƠ SỞ LÝ THUYẾT

### 2.14. Graph

- Đồ thị (Graph) là một cấu trúc dùng để mô hình hóa mối quan hệ giữa các đối tượng, bao gồm hai thành phần cơ bản gồm các đỉnh (các đối tượng) và các cạnh (liên kết giữa các cặp đỉnh, đại diện cho mối quan hệ hoặc đường kết nối giữa các đối tượng).
- Đồ thị cho phép tồn tại nhiều lộ trình và con đường khác nhau để di chuyển từ đỉnh này sang đỉnh khác, khác biệt hoàn toàn với tính chất tuần tự tuyến tính một chiều của mảng hay danh sách liên kết.

Thuật toán InitGraph(Tham chiếu Graph G, Số nguyên so\_dinh):

{ Đầu vào: Đồ thị G cần cấu hình, Số lượng đỉnh so\_dinh thực tế trên sa bàn }

{ Đầu ra: Đồ thị G được dọn dẹp và sẵn sàng kết nối các cạnh }

G.n <- so\_dinh

Cho i chạy từ 0 đến G.n - 1 thực hiện:

Cho j chạy từ 0 đến G.n - 1 thực hiện:

Nếu i bằng j thì:

G.matrix[i][j] <- 0      { Khoảng cách từ một điểm tới chính nó luôn bằng 0 }

Ngược lại:

G.matrix[i][j] <- INF    { Mặc định chưa có lối đi nào nối trực tiếp giữa i và j }

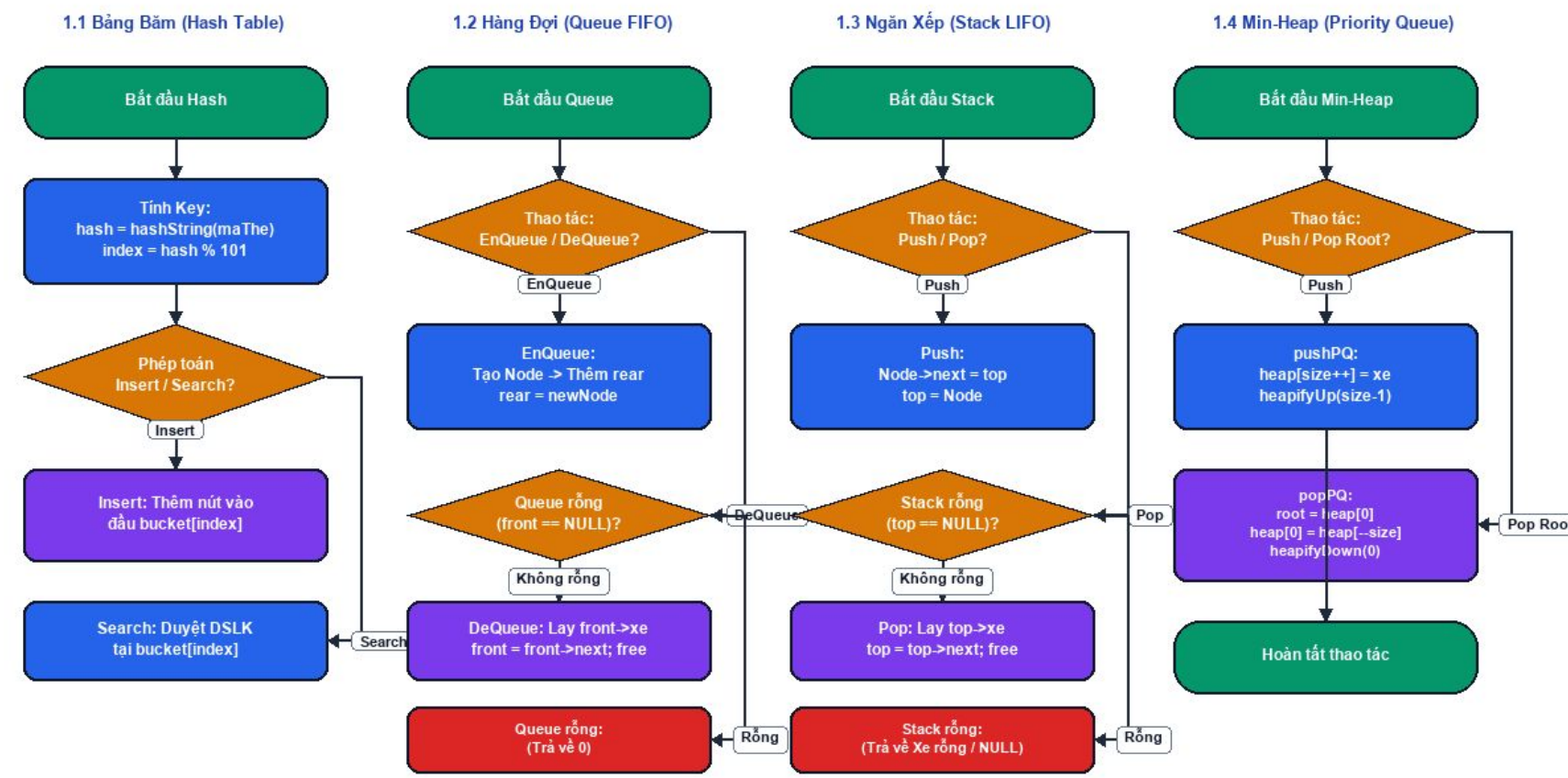
Kết thúc nếu

Kết thúc vòng lặp j

Kết thúc vòng lặp i

Kết thúc thuật toán

# III. MÔ HÌNH HOẠT ĐỘNG



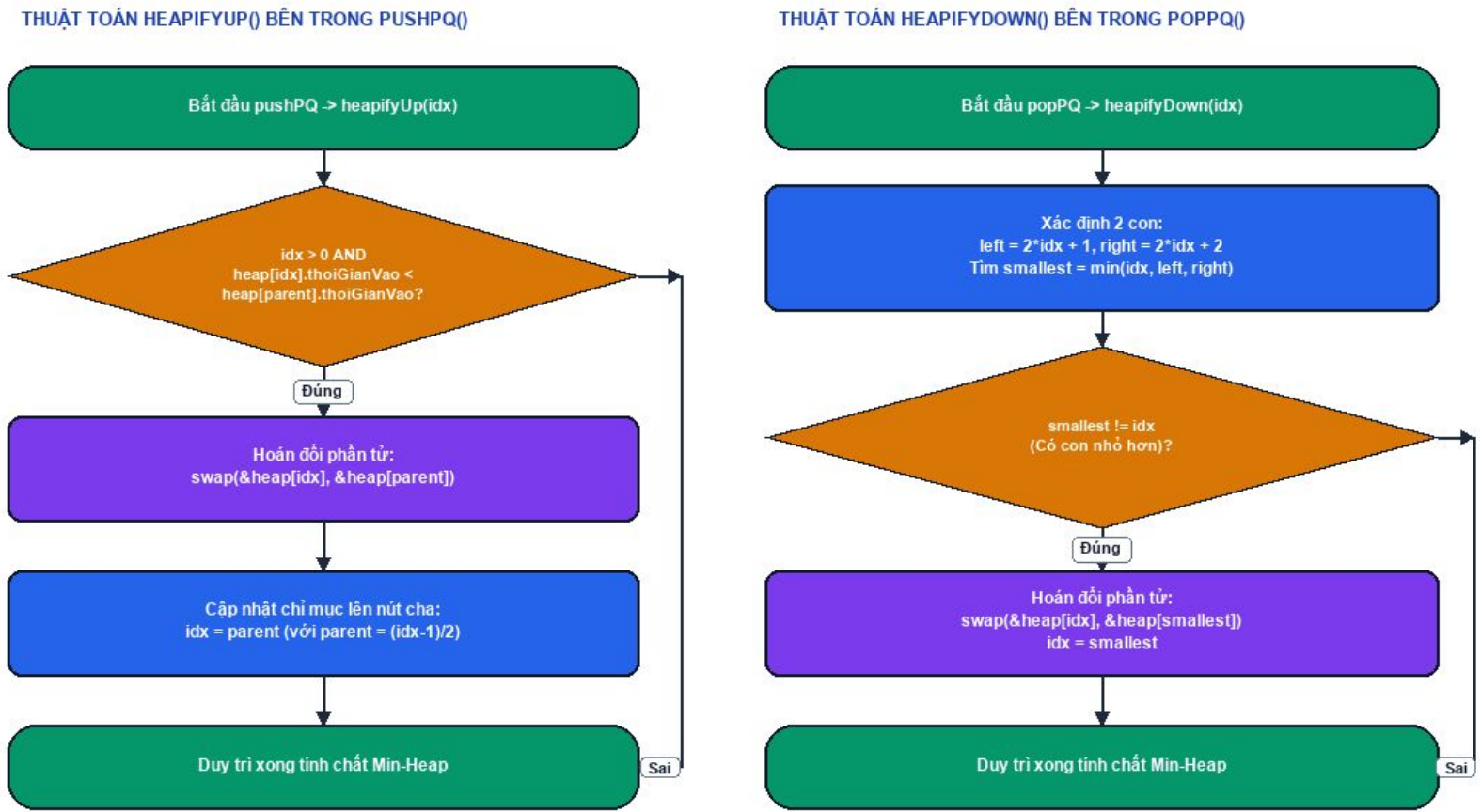
Hình 1. Lưu đồ giải thuật các cấu trúc dữ liệu cơ sở (HashTable, Queue FIFO, Stack LIFO, Min-Heap) 27

### III. MÔ HÌNH HOẠT ĐỘNG



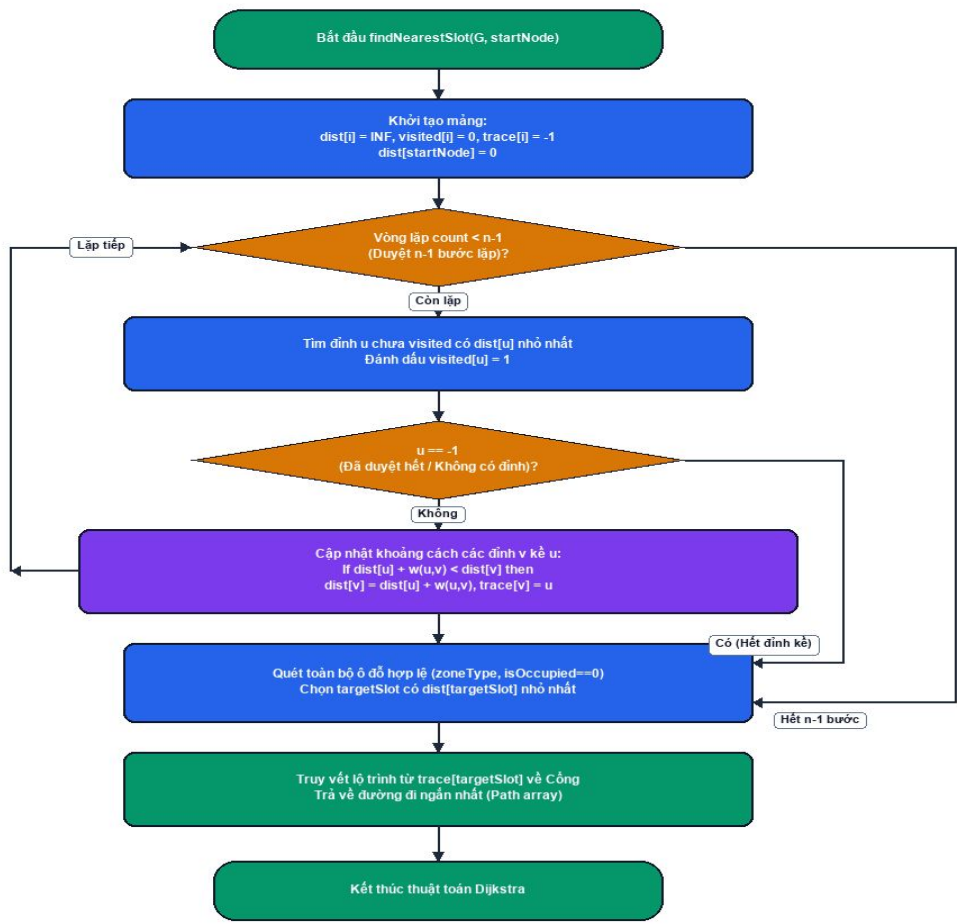
Hình 2. Lưu đồ giải thuật hàm InsertHash

# III. MÔ HÌNH HOẠT ĐỘNG



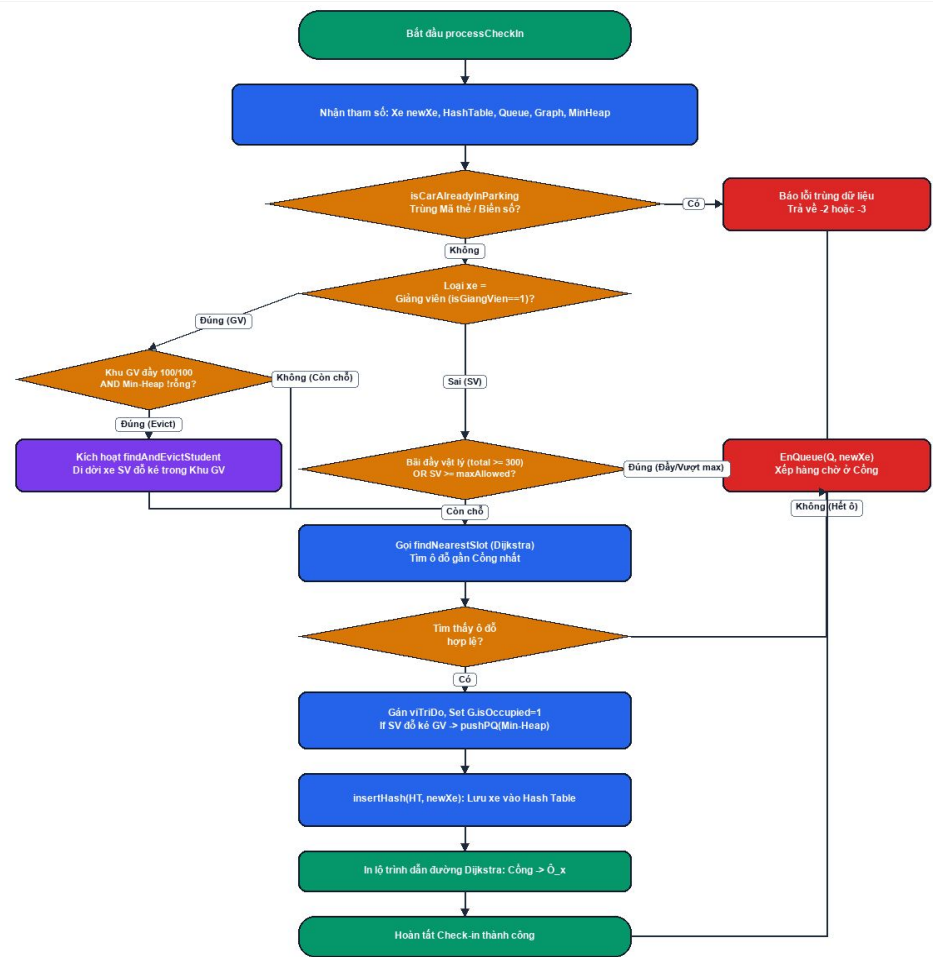
Hình 3. Lưu đồ giải thuật heapifyUp và heapifyDown

### III. MÔ HÌNH HOẠT ĐỘNG



Hình 4. Lưu đồ giải thuật `findNearestSlot`

# III. MÔ HÌNH HOẠT ĐỘNG



Hình 5. Lưu đồ giải thuật quy trình xe vào bãi (processCheckIn)

### III. MÔ HÌNH HOẠT ĐỘNG



Hình 6. Lưu đồ giải thuật findAndEvictStudent

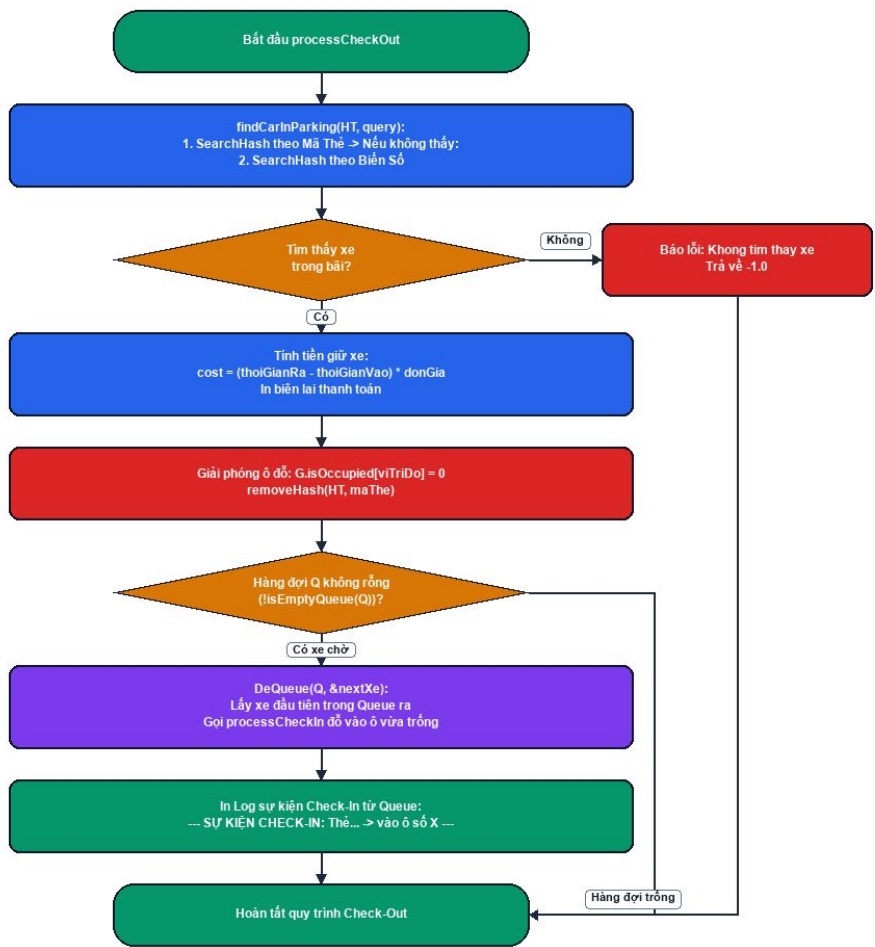


### III. MÔ HÌNH HOẠT ĐỘNG



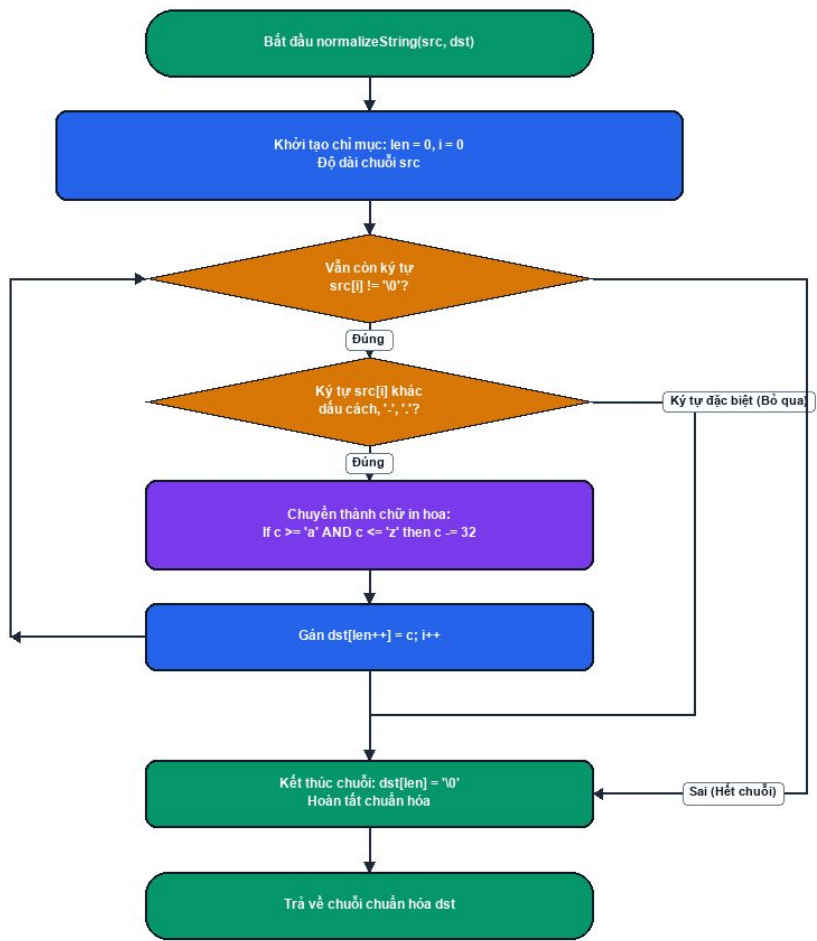
Hình 7. Lưu đồ giải thuật fuzzySearchLicensePlate

### III. MÔ HÌNH HOẠT ĐỘNG



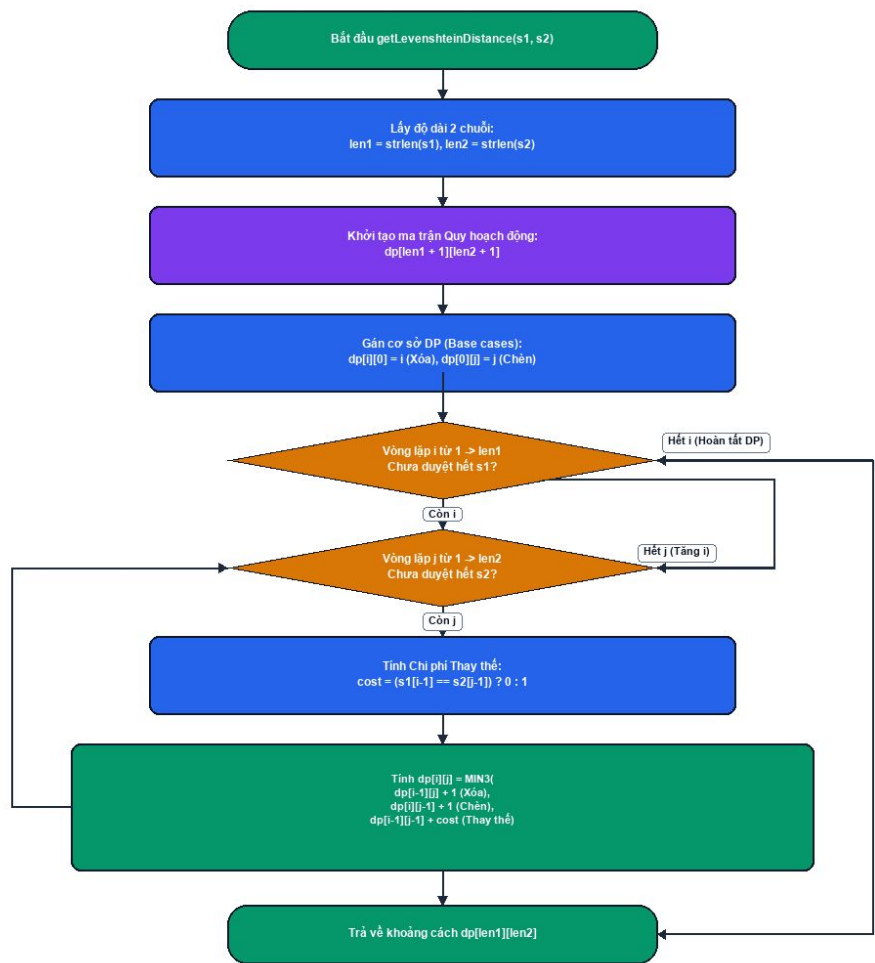
Hình 8. Lưu đồ giải thuật processCheckOut

### III. MÔ HÌNH HOẠT ĐỘNG



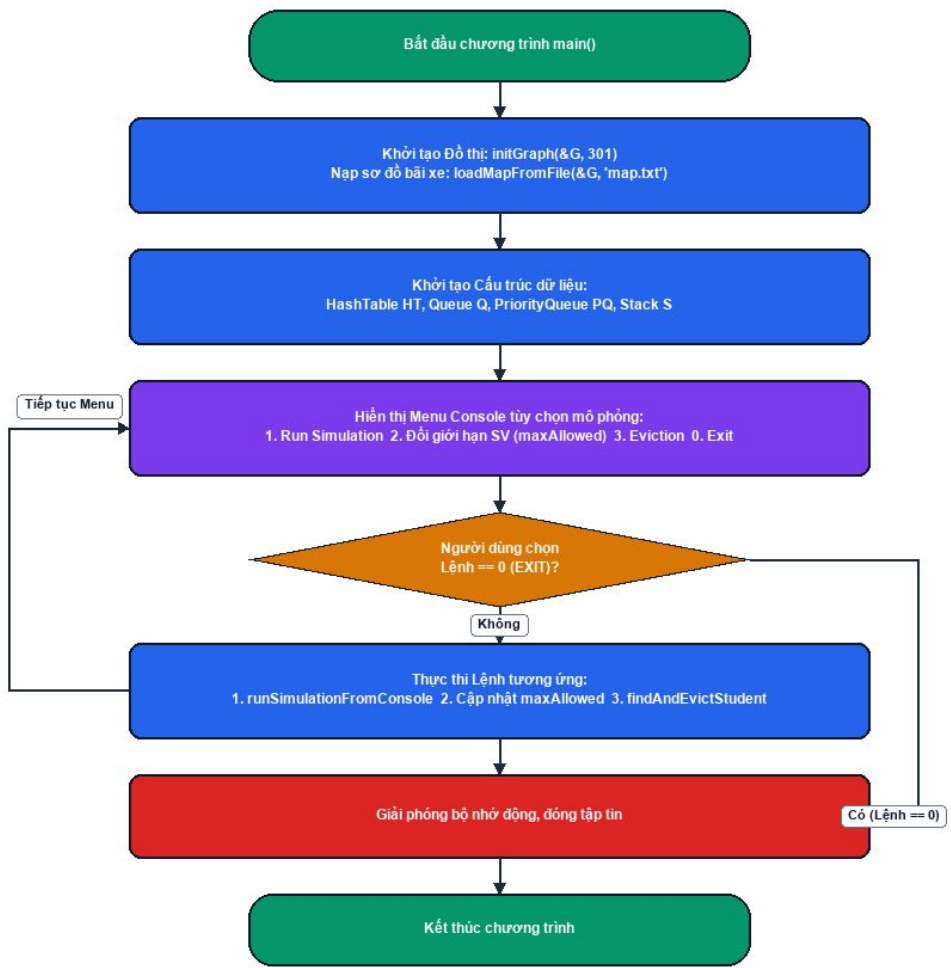
Hình 9. Lưu đồ giải thuật `normalizeString`

### III. MÔ HÌNH HOẠT ĐỘNG



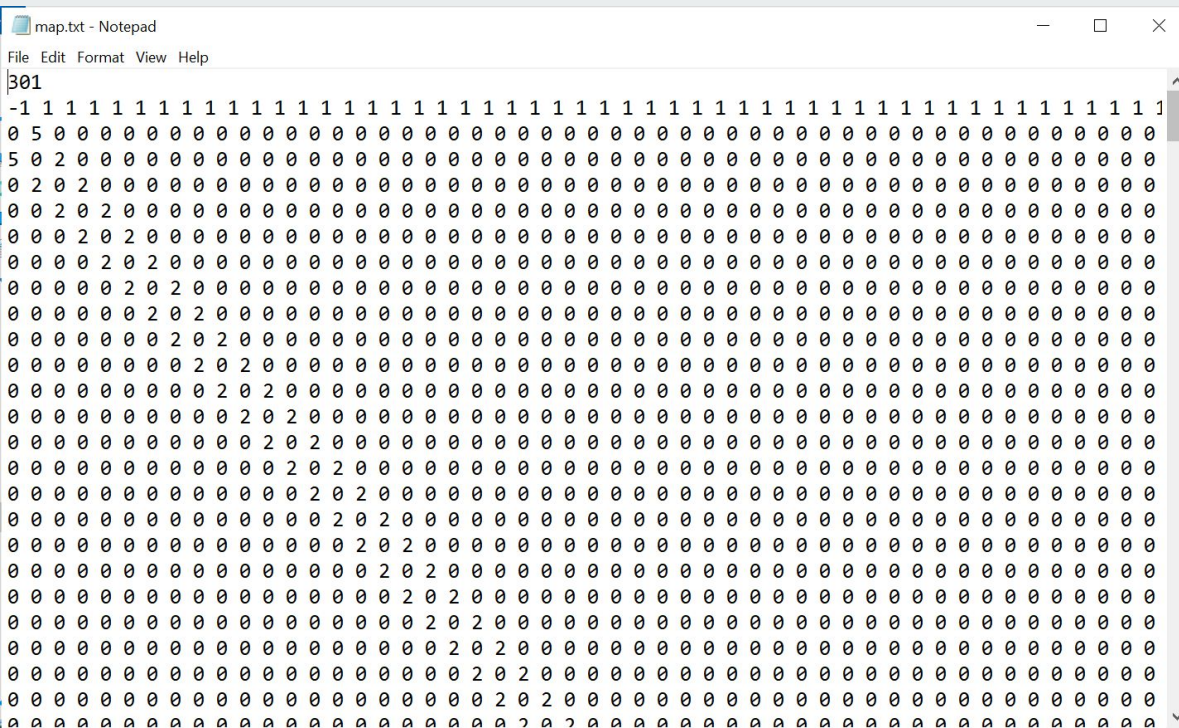
Hình 10. Lưu đồ giải thuật getLevenshteinDistance

### III. MÔ HÌNH HOẠT ĐỘNG



Hình 11. Lưu đồ giải thuật main()

## IV. KẾT QUẢ

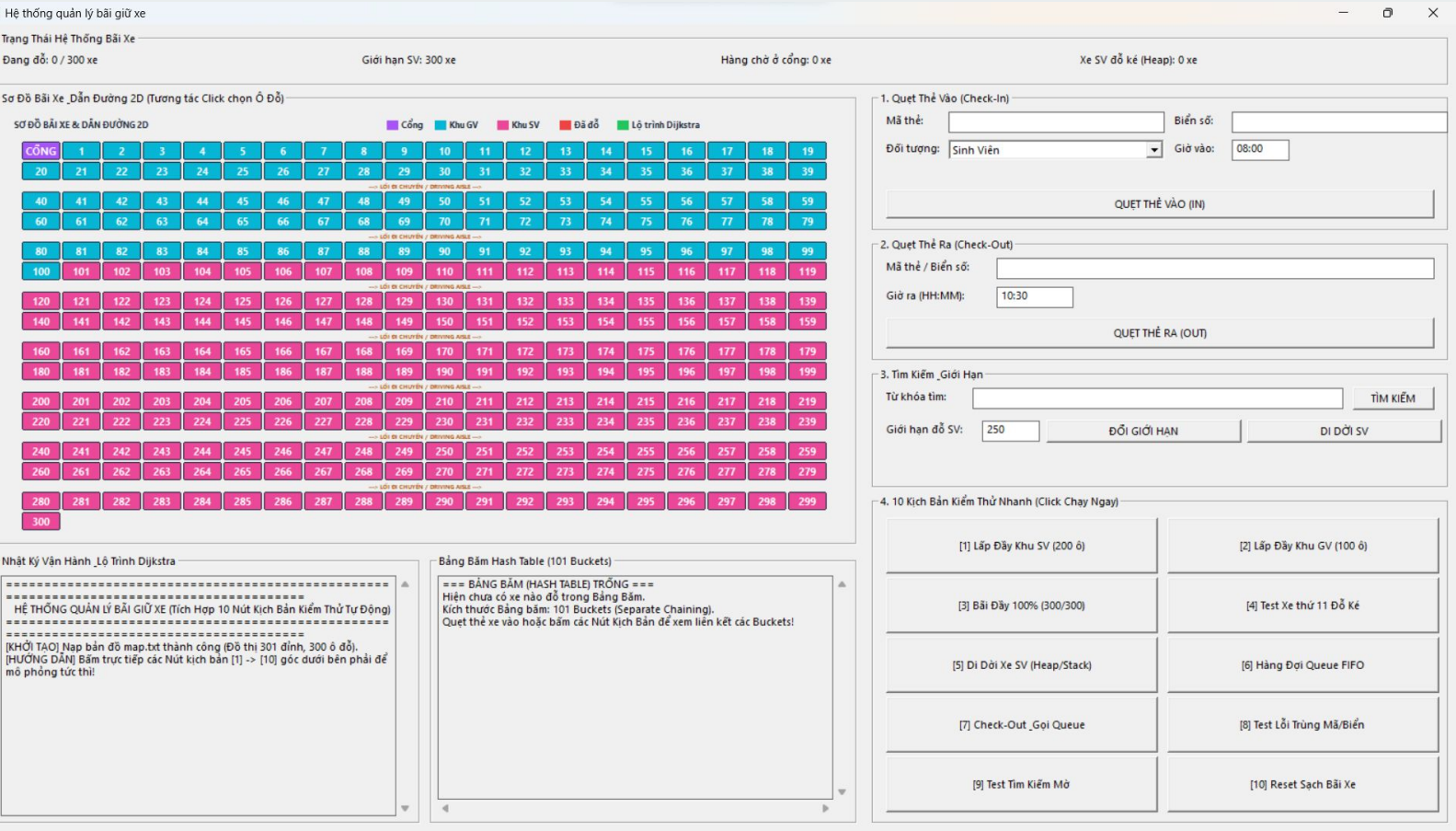


## IV. KẾT QUẢ

C:\Users\PC\Downloads\Báo cáo TH cấu trúc dữ liệu - chỉnh 3\26\_7\_2026\parking.exe

```
===== SMART CAMPUS PARKING =====  
Xe: 0 | Giới hạn SV: 300 | Số chỗ: 300  
1. Nhập sự kiện thu công (IN, OUT, FIND)  
2. Đổi giới hạn cho phép SV  
3. Di dời xe SV (khu GV)  
0. Thoát  
Chọn:
```

# IV. KẾT QUẢ



Hình 12. Giao diện tổng thể



Trang Thái Hệ Thống Bãi Xe

Đang đỗ: 200 / 300 xe

Giới hạn SV: 300 xe

Hàng chờ ở cổng: 0 xe

Xe SV đỗ kế (Heap): 0 xe

Sơ Đồ Bãi Xe & DẪN ĐƯỜNG 2D

Cổng

Khu GV

Khu SV

Đã đỗ

Lộ trình Dijkstra

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

Trang Thái Hệ Thống Bãi Xe

Đang đỗ: 200 / 300 xe

Giới hạn SV: 300 xe

Hàng chờ ở cổng: 0 xe

Xe SV đỗ kế (Heap): 0 xe

Sơ Đồ Bãi Xe & DẪN ĐƯỜNG 2D

Cổng

Khu GV

Khu SV

Đã đỗ

Lộ trình Dijkstra

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

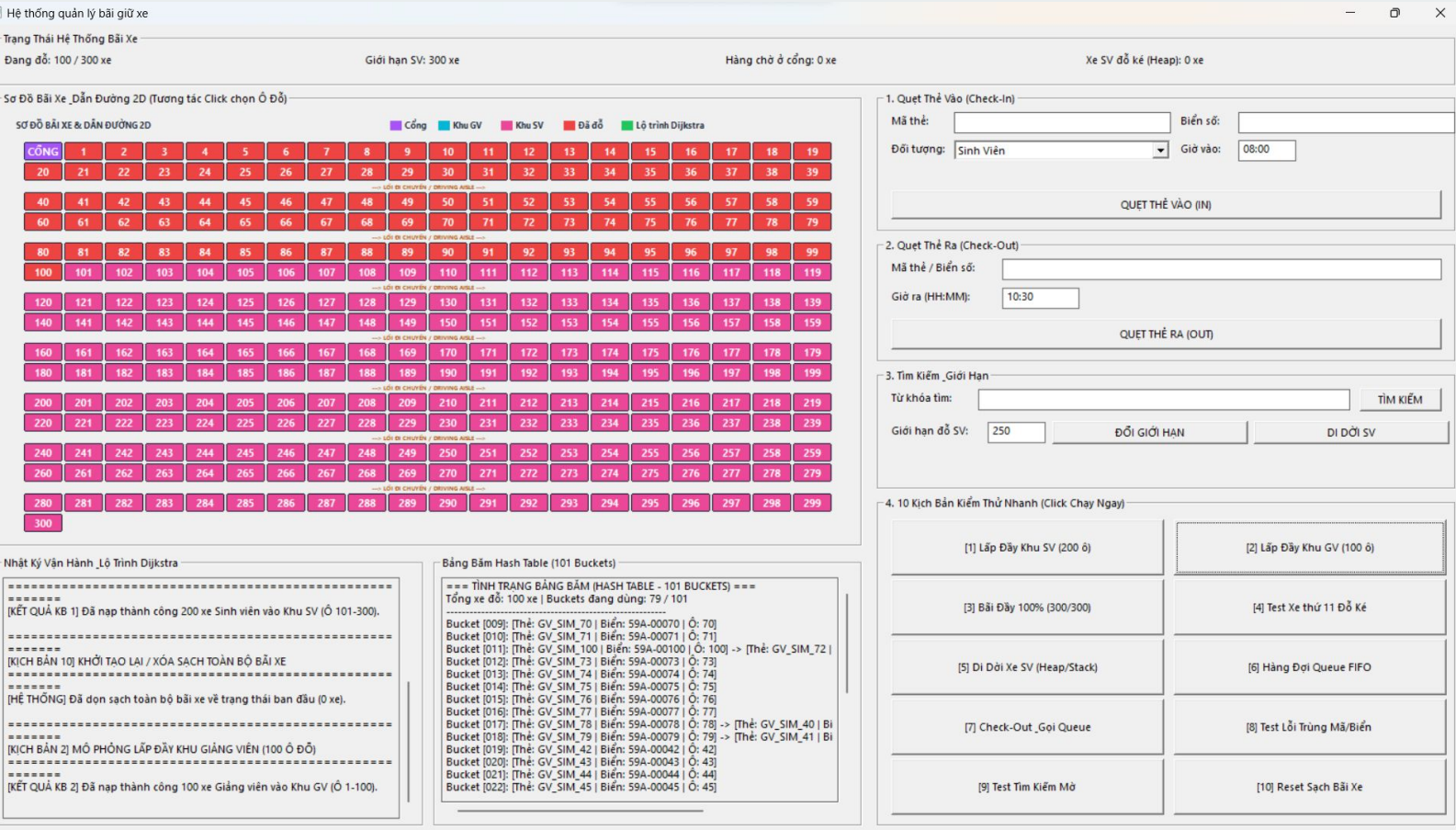
183

184

185

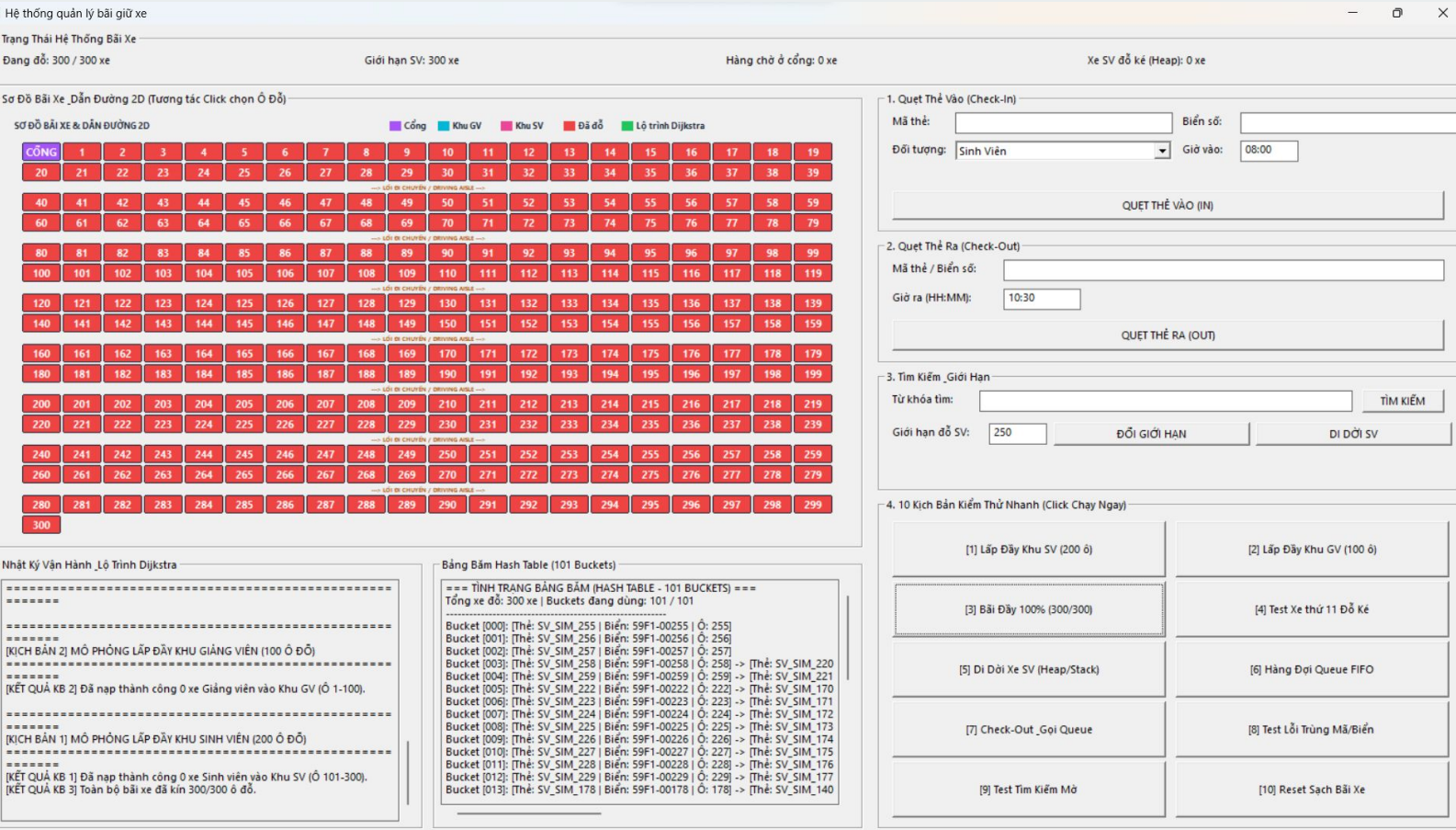
<

IV. KẾT QUẢ



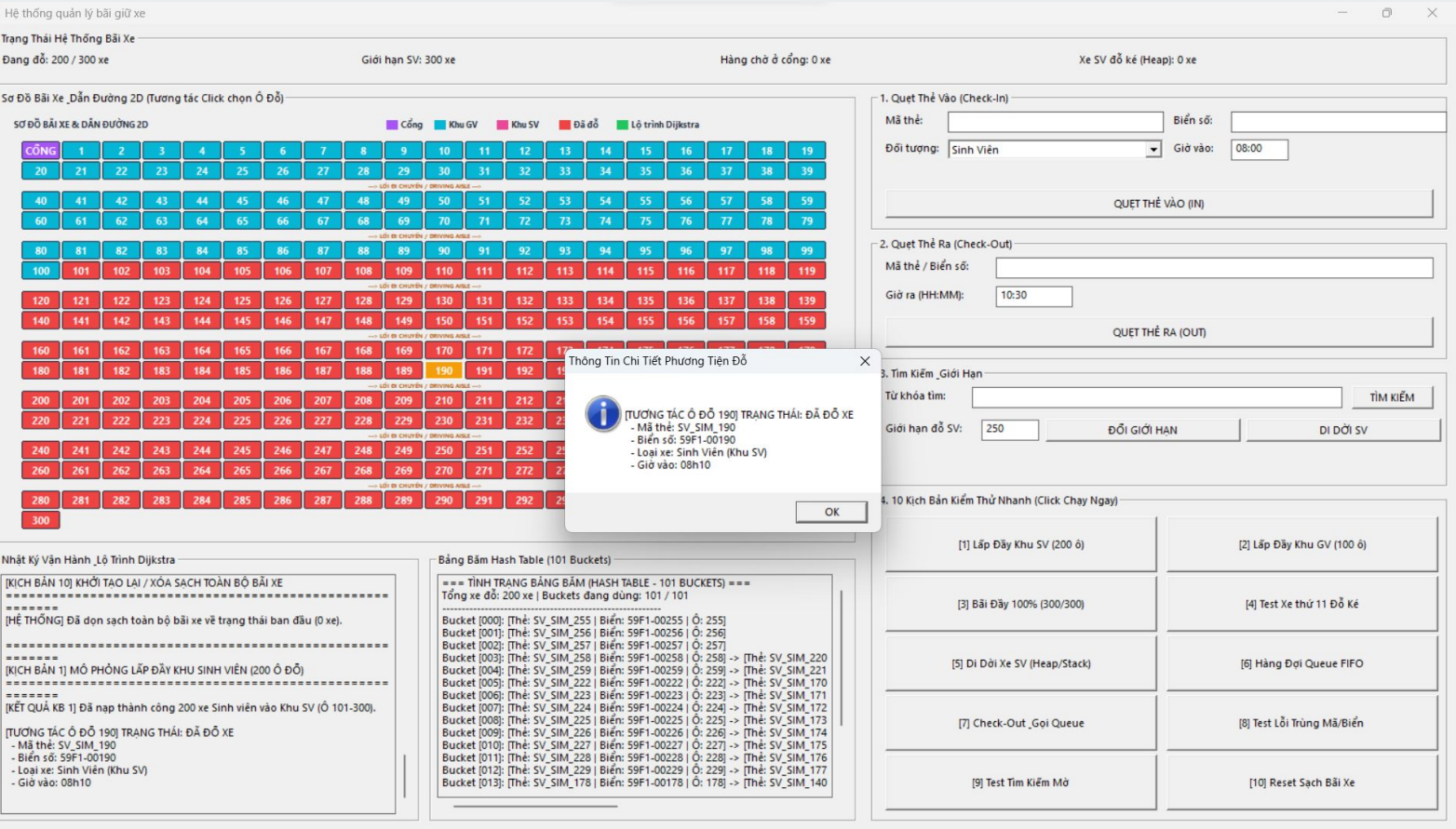
Hình 14. Kết quả khi bãi đỗ Giảng viên được lấp đầy

IV. KẾT QUẢ



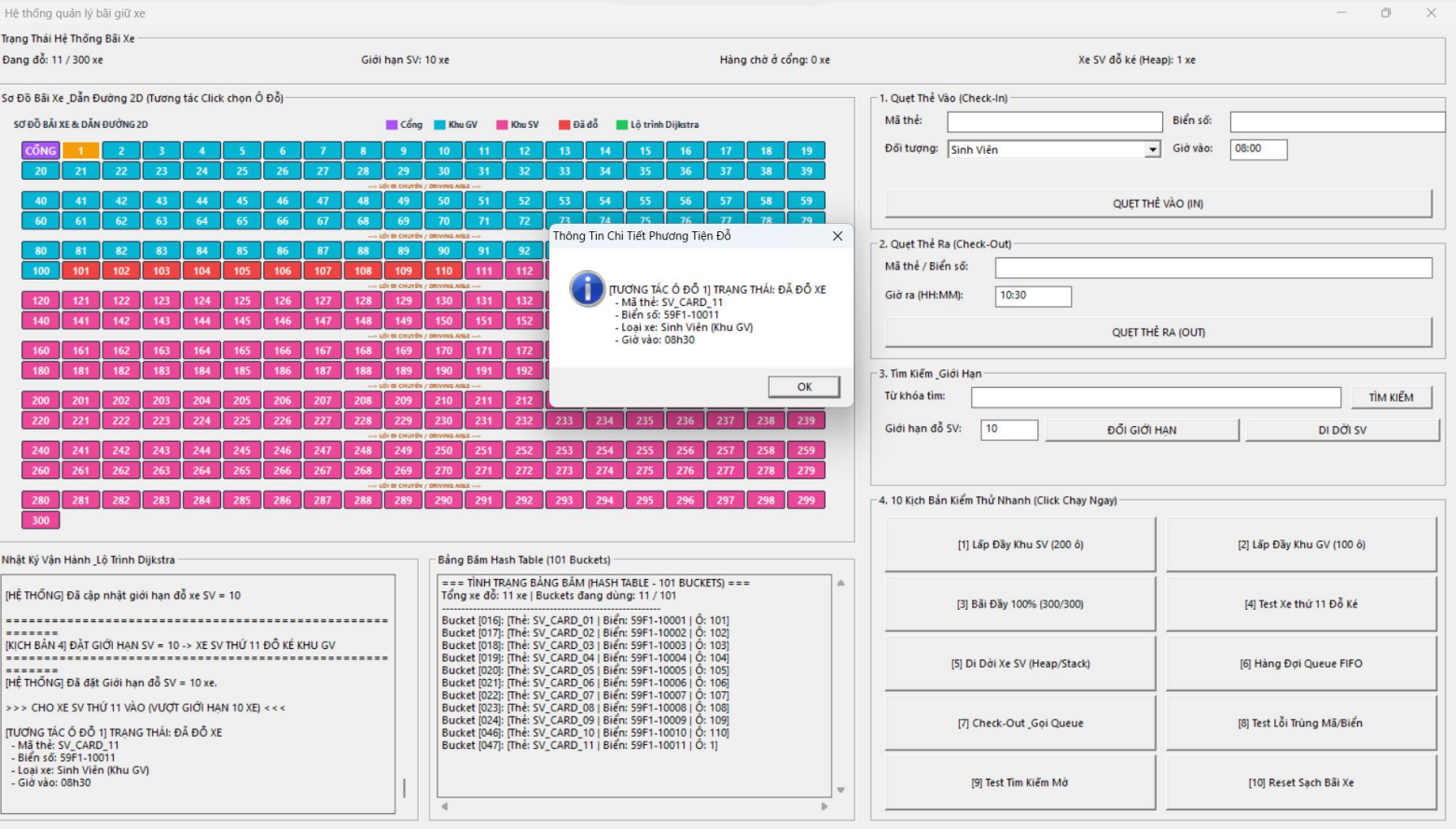


# IV. KẾT QUẢ



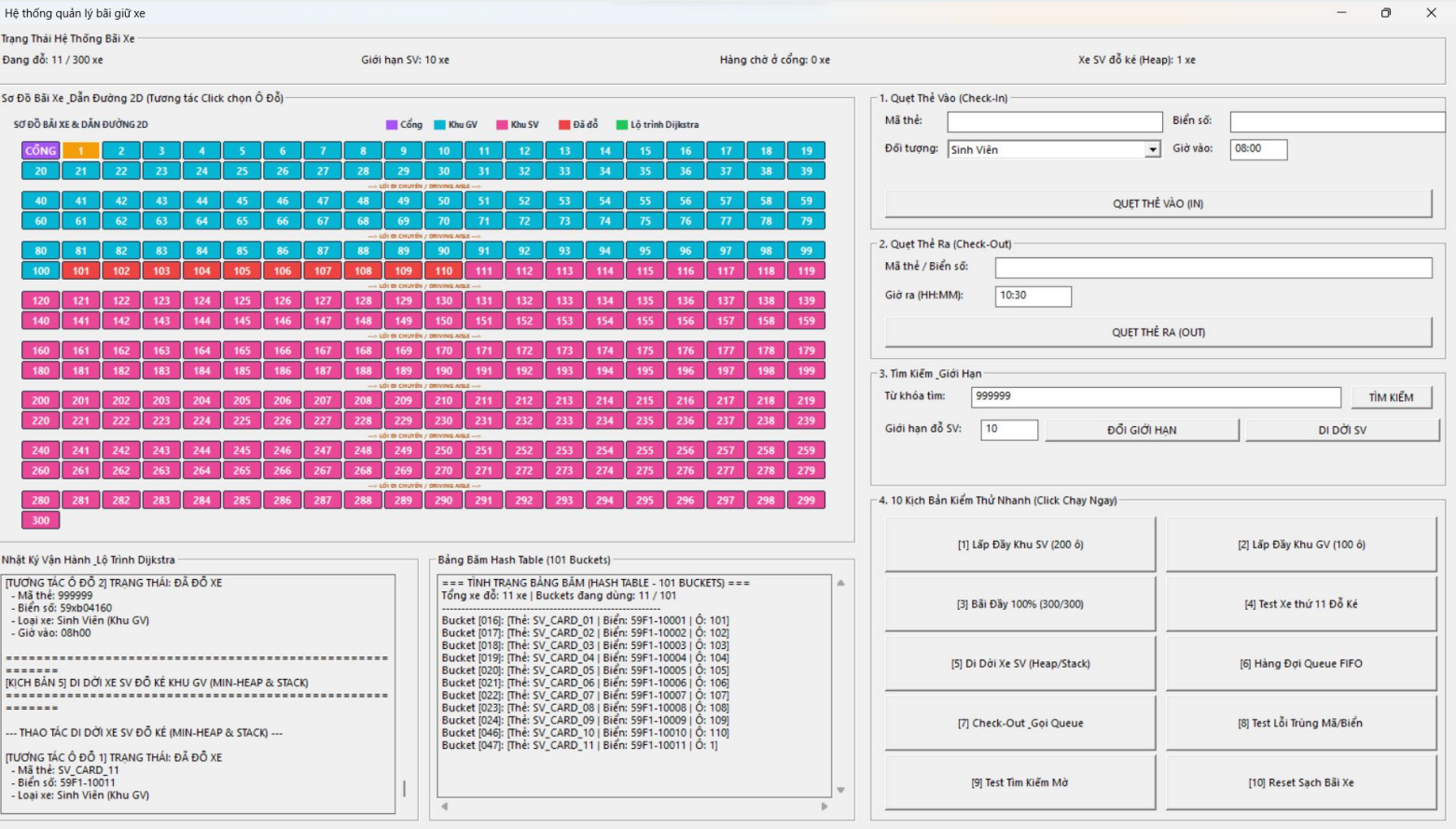
Hình 16. Khi bấm vào 1 ô bất kì thì sẽ có thông tin ô đỗ bao gồm mã thẻ, biển số xe, loại xe, khu đỗ và thời gian vào

# IV. KẾT QUẢ



Hình 17. Khi bãi xe sinh viên đầy thì xe sinh viên đậu vào bãi của GV

IV. KẾT QUẢ



Hình 18. Trường hợp bãi xe Giảng viên đã đầy thì phải di dời xe SV đợi nhờ để cho Giảng viên khác vào



Hệ thống quản lý bãi giữ xe  
 Trạng Thái Hệ Thống Bãi Xe  
 Đang đỗ: 14 / 300 xe  
 Giới hạn SV: 14 xe  
 Hàng chờ ở cổng: 0 xe  
 Xe SV đỗ kế (Heap): 4 xe

Sơ Đồ Bãi Xe & DẪN ĐƯỜNG 2D (Tương tác Click chọn Ô Đổ)  
 SƠ ĐỒ BÃI XE & DẪN ĐƯỜNG 2D  
 Cống Khu GV Khu SV Đã đổ Lộ trình Dijkstra

1. Quét Thẻ Vào (Check-In)  
 Mã thẻ:  Biển số:   
 Đối tượng:  Sinh Viên  Giờ vào:  08:00  
 QUÉT THẺ VÀO (IN)

2. Quét Thẻ Ra (Check-Out)  
 Mã thẻ / Biển số:   
 Giờ ra (HH:MM):  10:30  
 QUÉT THẺ RA (OUT)

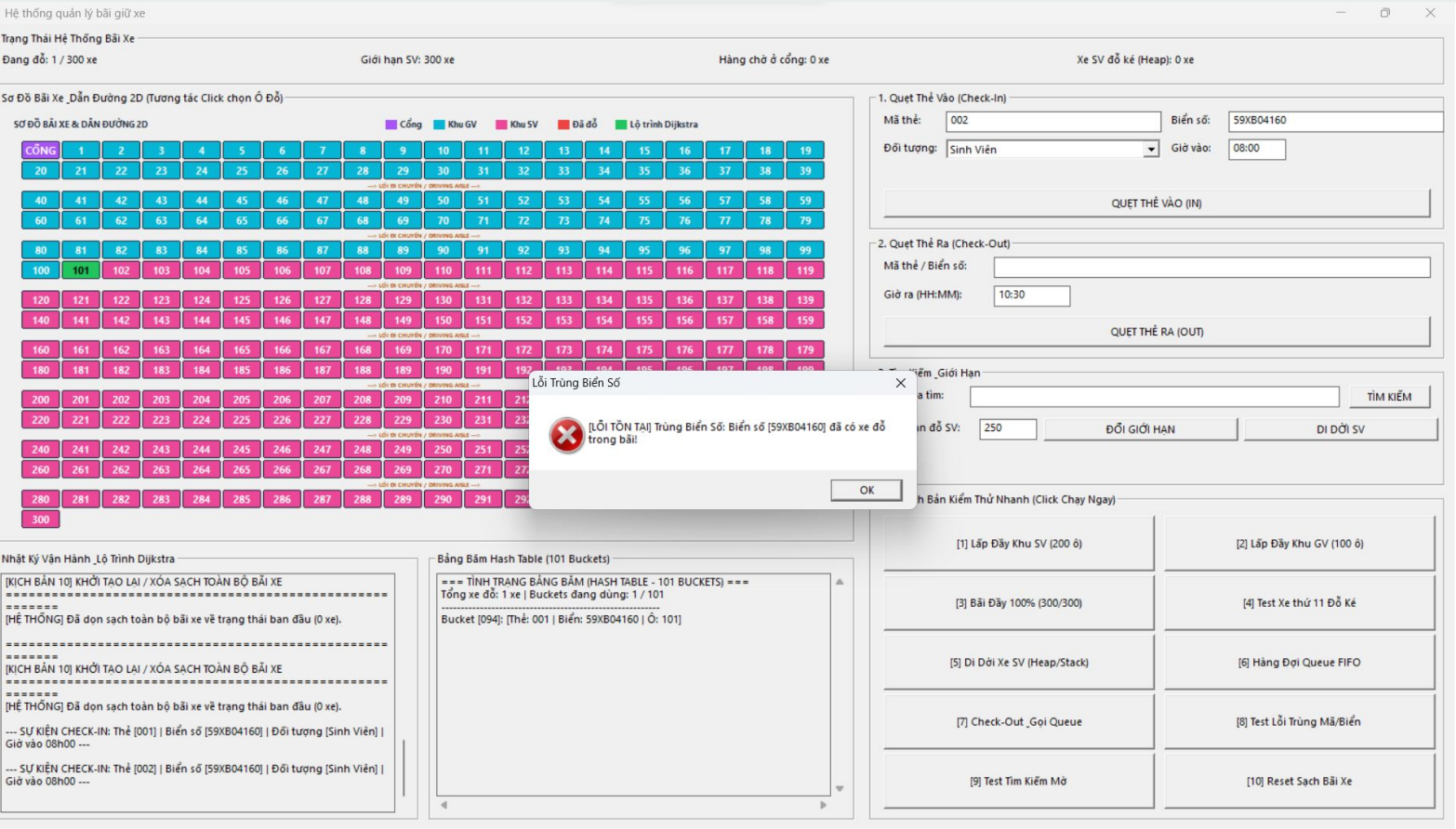
3. Tìm Kiếm\_Giới Hạn  
 Từ khóa tìm:  59F1-10001   
 Giới hạn đỗ SV:  10

4. 10 Kịch Bản Kiểm Thử Nhanh (Click Chạy Ngay)  
 [1] Lắp Đầy Khu SV (200 ô) [2] Lắp Đầy Khu GV (100 ô)  
 [3] Bãi Đầy 100% (300/300) [4] Test Xe thứ 11 ĐỖ Kế  
 [5] Di Dời Xe SV (Heap/Stack) [6] Hàng Đợi Queue FIFO  
 [7] Check-Out\_Gọi Queue [8] Test Lỗi Trùng Mã/Biến  
 [9] Test Tim Kiểm Mờ [10] Reset Sạch Bãi Xe

Nhật Ký Ván Hành\_Lộ Trình Dijkstra  
 101 | Giờ vào: 08h02  
 ---TÌM KIẾM THÔNG TIN BIẾN SỐ XE---  
 Tìm thấy 1 kết quả phù hợp cho từ khóa '59F1-10001':  
 [1] Thẻ: SV\_CARD\_01 | Biển số: 59F1-10001 | Loại xe: Sinh Viên (Khu SV) | Ô đỗ: 101 | Giờ vào: 08h02  
 ---TÌM KIẾM THÔNG TIN BIẾN SỐ XE---  
 Tìm thấy 1 kết quả phù hợp cho từ khóa '59F1-10001':  
 [1] Thẻ: SV\_CARD\_01 | Biển số: 59F1-10001 | Loại xe: Sinh Viên (Khu SV) | Ô đỗ: 101 | Giờ vào: 08h02  
 [TƯƠNG TÁC Ô ĐỖ 101] TRẠNG THÁI: ĐÃ ĐỖ XE  
 - Mã thẻ: SV\_CARD\_01  
 - Biển số: 59F1-10001  
 - Loại xe: Sinh Viên (Khu SV)  
 - Giờ vào: 08h02  
 OK

Bảng Băm Hash Table (101 Buckets)  
 === TÍNH TRẠNG BẢNG BĂM (HASH TABLE - 101 BUCKETS) ===  
 Tổng xe đỗ: 14 xe | Buckets đang dùng: 14 / 101  
 Thông Tin Chi Tiết Phương Tiện Đỗ  
 [TƯƠNG TÁC Ô ĐỖ 101] TRẠNG THÁI: ĐÃ ĐỖ XE  
 - Mã thẻ: SV\_CARD\_01  
 - Biển số: 59F1-10001  
 - Loại xe: Sinh Viên (Khu SV)  
 - Giờ vào: 08h02  
 OK

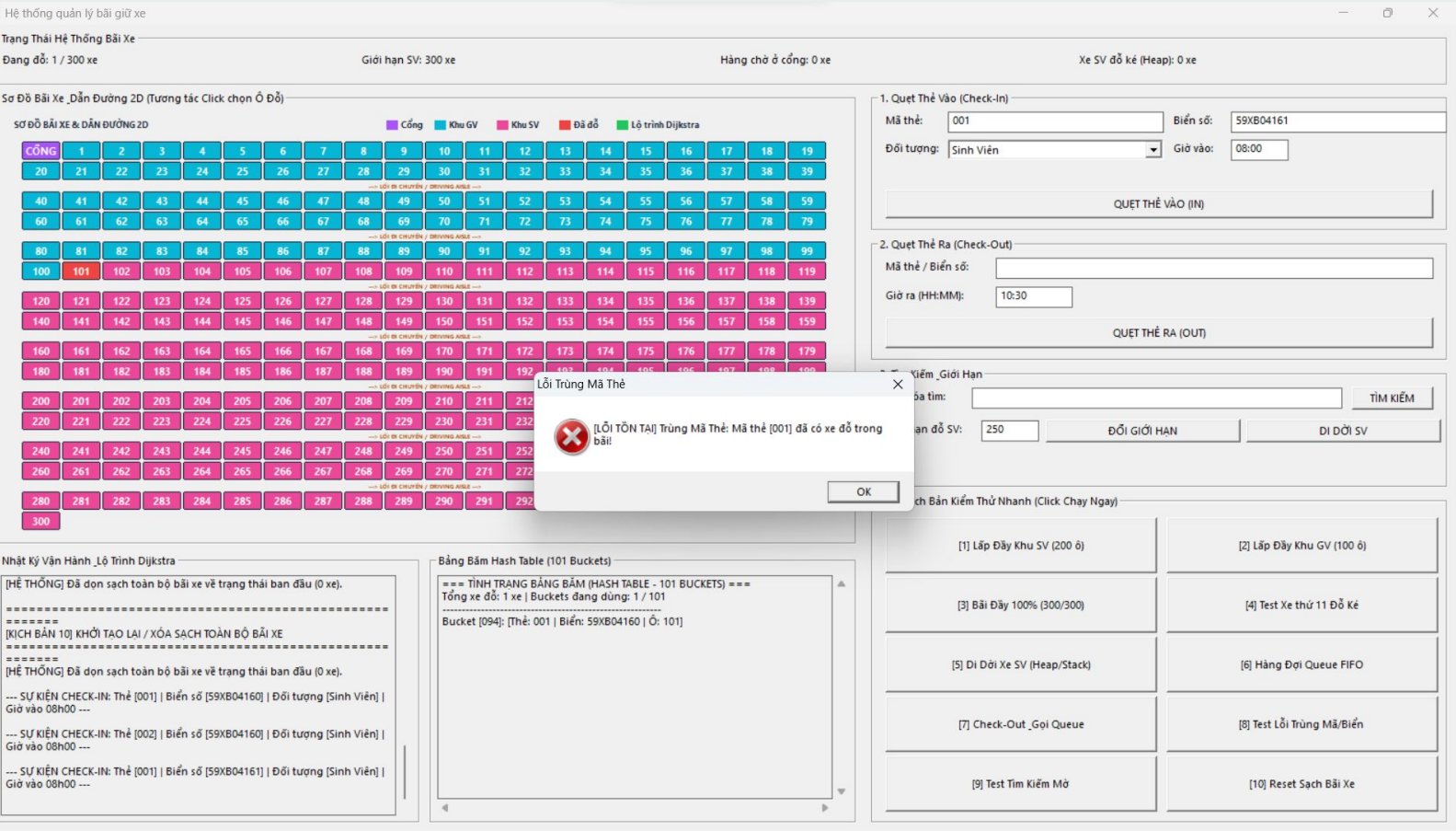
# IV. KẾT QUẢ



Hình 20. Thông báo trùng biển số



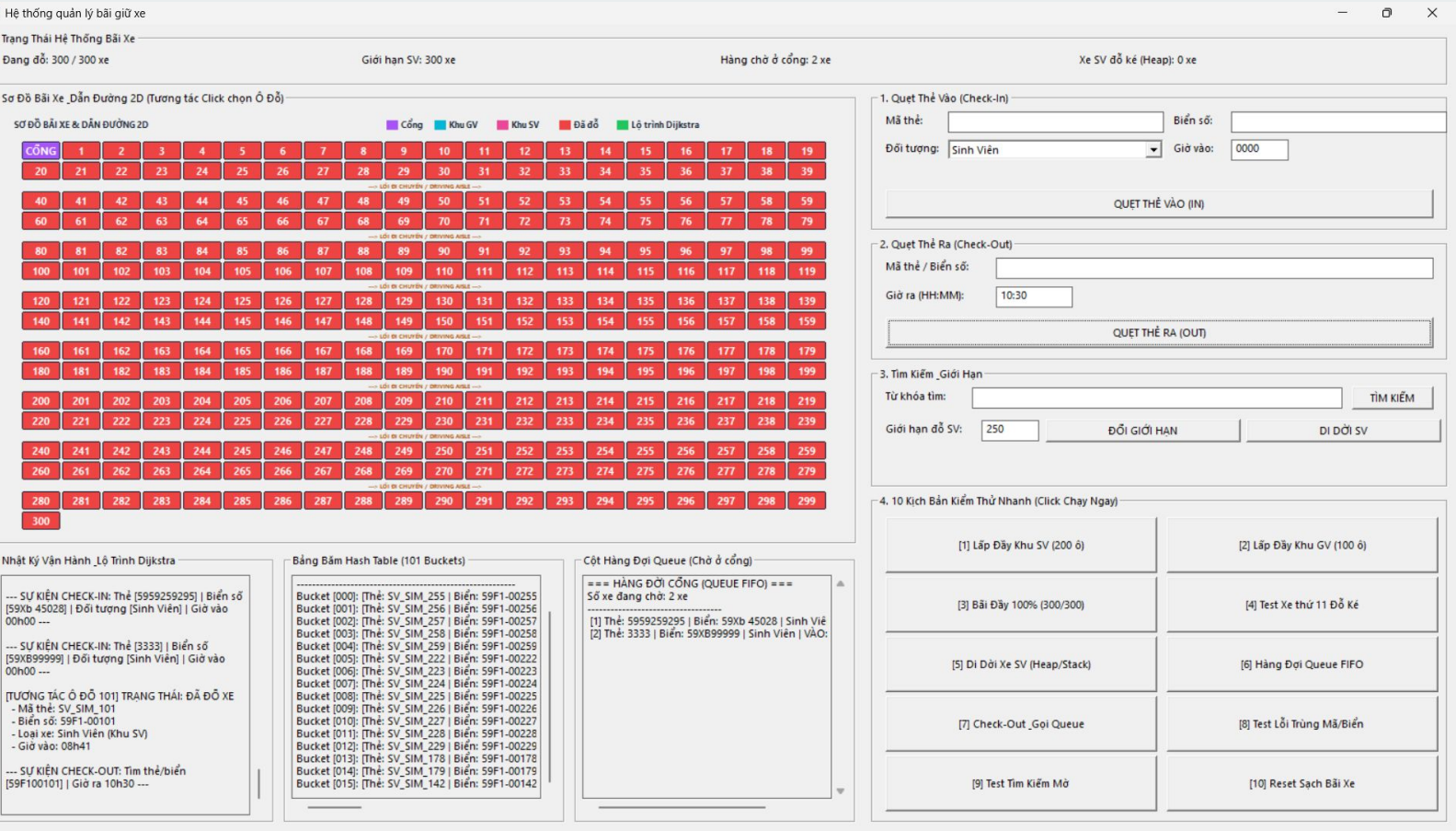
IV. KẾT QUẢ



Hình 21. Thông báo trùng mã thẻ

50

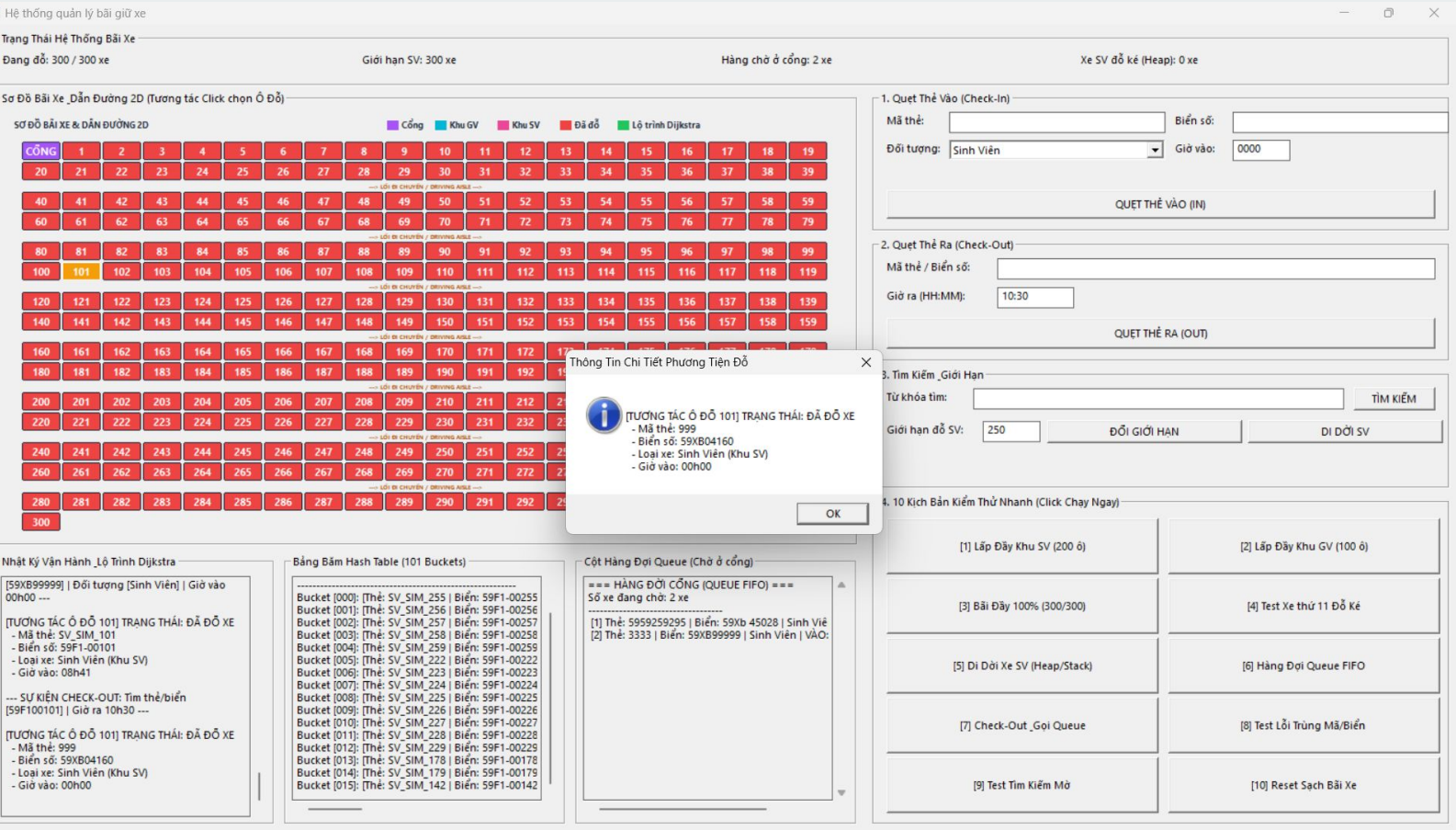
IV. KẾT QUẢ



Hình 23. Khi có một xe trong bãi ra khỏi bãi đỗ thì xe trong hàng đợi sẽ được đưa vào bãi xe theo FIFO



# IV. KẾT QUẢ



Hình 24. Xe mang biển số 59XB04160 vào ô 101

## IV. PHÂN TÍCH ĐÁNH GIÁ

### 1. Lưu trữ trung tâm (Bảng băm & Separate Chaining)

- Khởi tạo bảng mảng tĩnh gồm 101 ô chứa (buckets).
- Giải quyết xung đột bộ nhớ bằng Danh sách liên kết đơn (Separate Chaining), đảm bảo toàn vẹn dữ liệu.
- Tối ưu thời gian tra cứu và cung cấp thông số sức chứa theo thời gian thực.

### 2. Điều tiết giao thông công vào (Hàng đợi - Queue)

- Hoạt động như vùng đệm an toàn chống quá tải (hệ thống tối đa N xe).
- EnQueue: Tự động đưa xe vào trạng thái chờ khi bãi đạt giới hạn.
- DeQueue: Lấy xe khỏi hàng đợi để phân bổ chỗ đỗ ngay khi có vị trí trống.

### 3. Số hóa không gian & Định tuyến (Đồ thị & Dijkstra)

- Không gian bãi đỗ được mô phỏng bằng Ma trận kề, trạng thái trống (isOccupied) và loại phân khu (zoneType).
- Thuật toán Dijkstra đóng vai trò lõi điều hướng, tìm lộ trình di chuyển ngắn nhất từ công.
- Tích hợp cơ chế Early Stop, kết thúc duyệt đỉnh ngay khi tìm được ô đỗ đúng đối tượng và còn trống.

## IV. PHÂN TÍCH ĐÁNH GIÁ

### 4. Quản lý xe đỗ tạm (Priority Queue / Min-Heap)

- Kích hoạt khi sinh viên phải đỗ nhờ phân khu Giảng viên do hết chỗ.
- Cấu trúc Min-Heap liên tục vun đống (Heapify), đẩy phương tiện vào bãi sớm nhất lên đỉnh (Root).
- Hỗ trợ trích xuất chính xác phương tiện cần giải tỏa khi có Giảng viên tiến vào bãi.

### 5. Cơ chế dọn đường (Ngăn xếp - Stack)

- Vận hành theo nguyên lý LIFO để dời các phương tiện cản đường xe mục tiêu.
- Push: Tạm thời lưu trữ các phương tiện cản đường vào bộ nhớ Ngăn xếp.
- Pop: Khôi phục chính xác toàn bộ phương tiện về vị trí ban đầu trên Đồ thị.

### 6. So khớp biến số lỗi (Fuzzy Search & Levenshtein)

- Xử lý sự cố sai lệch biến số trong quá trình Check-out.
- Dùng Quy hoạch động (DP) tính khoảng cách Levenshtein để đo lường mức độ sai lệch ký tự.
- Lọc các ứng viên có sai số  $\leq 3$  và dùng Bubble Sort để trích xuất 3 phương tiện có độ tương đồng cao nhất.

**CẢM ƠN MỌI NGƯỜI ĐÃ THEO DÕI**